

Docker入门及精通笔记

 乐乐 _ 分类: [学习笔记](#) 浏览: 775 评论: 3 ☒ 字体 ☐ 皮肤 ☐ 左侧展开

最后修改于: 2021/03/01 19:06:31

[展开目录](#)



镜像 (image) 模板, 通过模板创建容器服务, 镜像>run>tomcat01容器 (提供服务器), 通过镜像创建多个容器 (服务运行在容器中)
容器 (container)
仓库 (repository)

安装docker

查询已安装的docker并删除,更新yum包

1. `yum list installed|grep docker`
2. `yum remove docker-ce.x86_64`
3. `yum update`



1.安装一些必要的软件

1. `sudo yum install -y yum-utils device-mapper-persistent-data lvm2`



2.添加软件源信息

1. `sudo yum-config-manager --add-repo https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo`



3.更新并安装Docker-CE

1. `sudo yum makecache fast`
2. `sudo yum -y install docker-ce docker-ce-client containerd.io`



注: ce是社区版, ee是企业版

4.开启Docker服务

1. `sudo service docker start`



5.查看是否安装成功

1. `docker version`



6.docker run

1. `docker run hello-world`



```
[root@localhost ~]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
0e03bdcc26d7: Pull complete
Digest: sha256:31b9c7d48790f0d8c50ab433d9c3b7e17666d6993084c002c2ff1ca09b96391d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

无镜像，pull拉取

🏆 7.查看镜像

- 1. `docker images`



🏆 #了解卸载docker

1.删除依赖

- 1. `yum remove docker-ce docker-ce-cli containerd.io`



2.删除资源

- 1. `rm -rf /var/lib/docker`
- 2.
- 3. `#/var/lib/docker` *docker默认工作路径*



#

- 1. 注意：
- 2. # 官方软件源默认启用了最新的软件，您可以通过编辑软件源的方式获取各个版本的软件包。例如官方并没有将测试版本的软件源置为可用，您可以通过以下方式开启。同理可以开启各种测试版本等。
- 3. # `vim /etc/yum.repos.d/docker-ee.repo`
- 4. # 将[`docker-ce-test`]下方的`enabled=0`修改为`enabled=1`
- 5. #
- 6. # 安装指定版本的Docker-CE:
- 7. # Step 1: 查找Docker-CE的版本:
- 8. # `yum list docker-ce.x86_64 --showduplicates | sort -r`
- 9. # *Loading mirror speeds from cached hostfile*
- 10. # *Loaded plugins: branch, fastestmirror, langpacks*
- 11. # `docker-ce.x86_64` `17.03.1.ce-1.el7.centos` `docker-ce-stable`
- 12. # `docker-ce.x86_64` `17.03.1.ce-1.el7.centos` `@docker-ce-stable`
- 13. # `docker-ce.x86_64` `17.03.0.ce-1.el7.centos` `docker-ce-stable`
- 14. # *Available Packages*
- 15. # Step2: 安装指定版本的Docker-CE: (VERSION例如上面的`17.03.0.ce.1-1.el7.centos`)
- 16. # `sudo yum -y install docker-ce-[VERSION]`



🏆 8.设置阿里云镜像加速

通过修改daemon配置文件/etc/docker/daemon.json来使用加速器

☰ 阿里云

容器镜像服务

▼ 默认实例

镜像仓库

命名空间

授权管理

代码源

访问凭证

▶ 企业版实例

▼ 镜像中心

镜像搜索

我的收藏

镜像加速器

镜像加速器

加速器

使用加速器可以提升获取Docker官方镜像的速度

加速器地址

https://emgy1n9y.mirror.aliyuncs.com 复制

操作文档

Ubuntu CentOS Mac Windows

1. 安装 / 升级Docker客户端

推荐安装 1.10.0 以上版本的Docker客户端，参考文档 [docker-ce](#)

2. 配置镜像加速器

针对Docker客户端版本大于 1.10.0 的用户

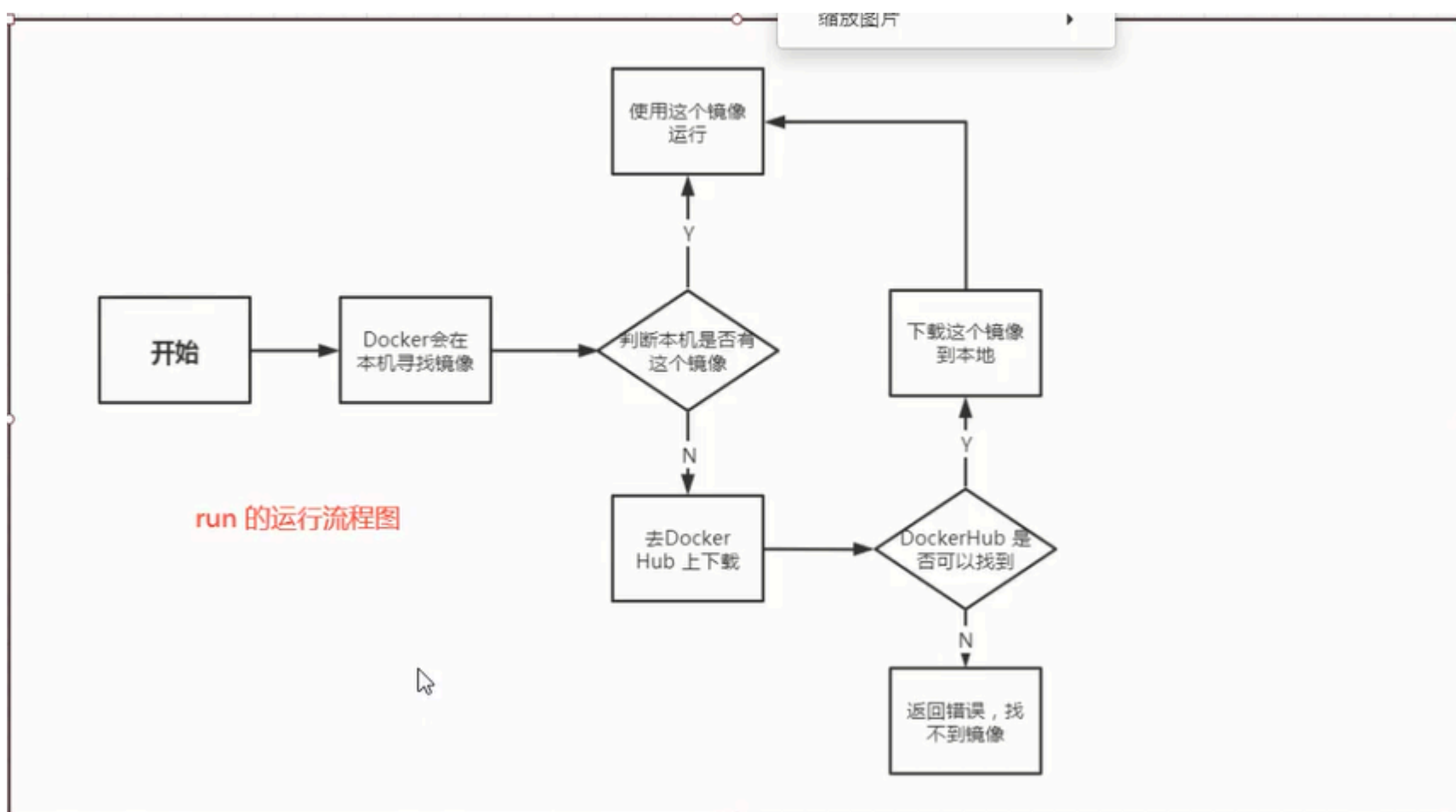
您可以通过修改daemon配置文件 `/etc/docker/daemon.json` 来使用加速器

```
sudo mkdir -p /etc/docker
```

```
1. sudo mkdir -p /etc/docker
2.
3. sudo tee /etc/docker/daemon.json <<-'EOF'
4.
5. {
6.
7.   "registry-mirrors": ["https://emgy1n9y.mirror.aliyuncs.com"]
8.
9. }
10.
11. EOF
12.
13. sudo systemctl daemon-reload
14.
15. sudo systemctl restart docker
```



🏆 docker run原理

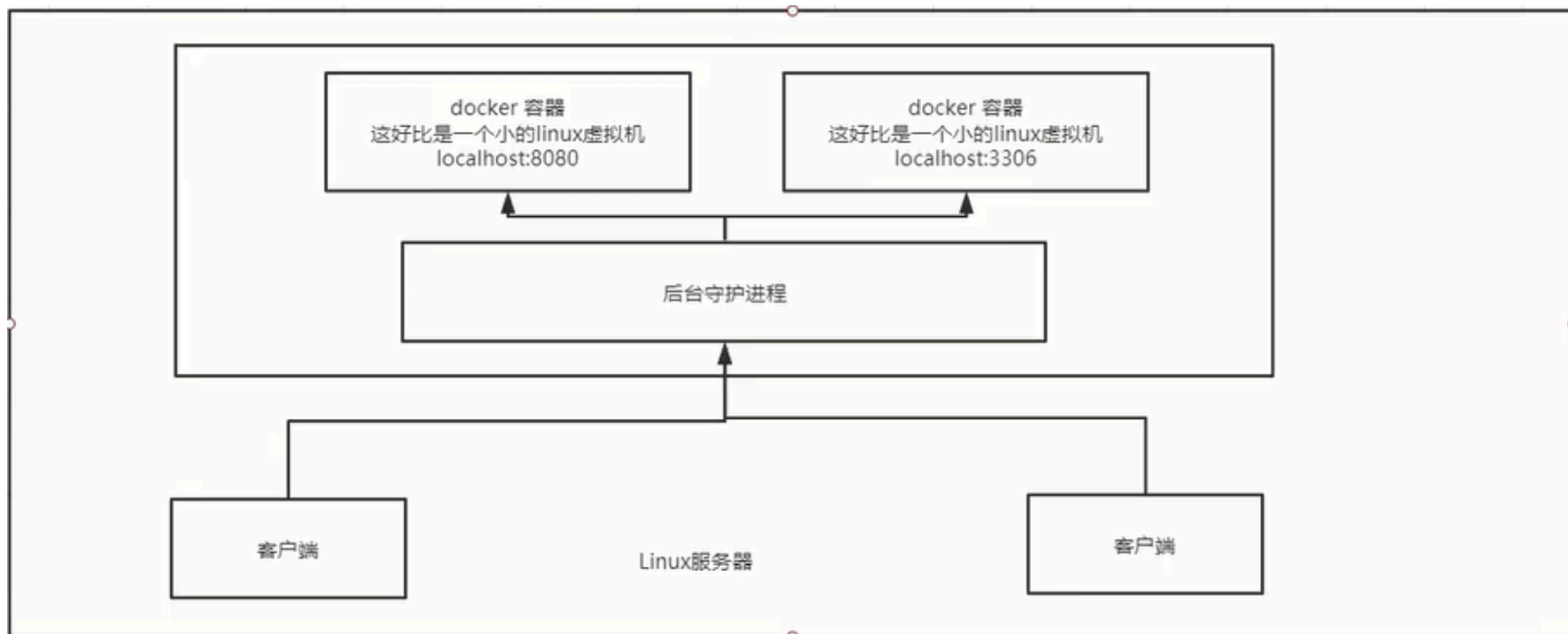


🏆 底层原理

docker是怎样工作的?

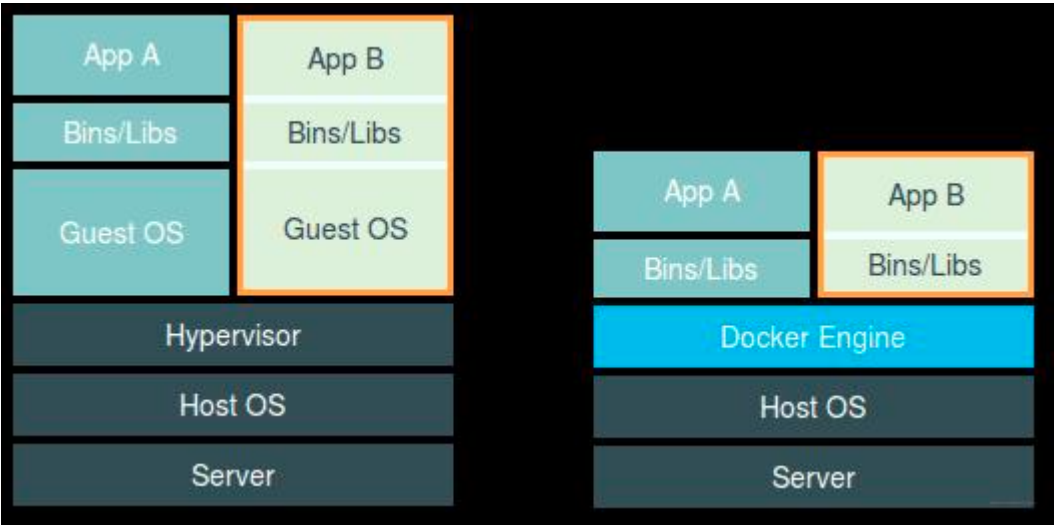
docker是一个C-S架构的系统，docker的守护进程运行在主机上，通过socket从客户端访问

docker server接收到docker client的命令，就会执行这个命令



docker为什么比vm快

1.docker有着比vm更少的抽象层



2.docker利用的是宿主机的内核，vm需要的是guest os

vm是加载guest os， docker是基于宿主机操作系统

docker常用命令

帮助命令

1. `docker version` *#现实docker的版本信息*
2. `docker info` *#显示docker系统信息，包括镜像和容器的数量*
3. `docker --help`

镜像命令

docker images

docker images 是查看所有本的主机上的镜像

1. `[root@localhost docker]# docker images`
2.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	bf756fb1ae65	12 months ago	13.3kB
- 3.
4. *#解释*
5. REPOSITORY 镜像的仓库源
6. TAG 镜像的标签
7. IMAGE ID 镜像的id
8. CREATED 镜像的创建时间
9. SIZE 镜像的大小
10. *#可选项*
11. `-a, --all` *#列出所有镜像*
12. `-q, --quiet` *#只显示镜像id*

docker search

docker search 搜索镜像

1. `[root@localhost docker]# docker search mysql`
2.

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
mysql	MySQL is a widely used, open-source relation...	10418	[OK]	
mariadb	MariaDB is a community-developed fork of MyS...	3866	[OK]	
- 3.
- 4.
- 5.
- 6.
7. *#可选项，可以通过搜索来过滤*
8. `--filter=STARS=3000` *#搜索出来的镜像就是STARS大于3000的*
9. `[root@localhost docker]# docker search mysql --filter=STARS=4000`
10.

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
mysql	MySQL is a widely used, open-source relation...	10418	[OK]	
- 11.

docker pull

docker pull 下载镜像

```
1. # 下载镜像 docker pull 镜像名[:tag]
2. [root@localhost docker]# docker pull mysql
3. Using default tag: latest #如果不写tag, 默认是最新
4. latest: Pulling from library/mysql
5. a076a628af6f: Pull complete # 分层下载, docker image的核心, 联合文件系统
6. f6c208f3f991: Pull complete
7. 88a9455a9165: Pull complete
8. 406c9b8427c6: Pull complete
9. 7c88599c0b25: Pull complete
10. 25b5c6debda7: Pull complete
11. 43a5816f1617: Pull complete
12. 1a8c919e89bf: Pull complete
13. 9f3cf4bd1a07: Pull complete
14. 80539cea118d: Pull complete
15. 201b3cad54ce: Pull complete
16. 944ba37e1c06: Pull complete
17. Digest: sha256:feada149cb8ff54eade1336da7c1d080c4a1c7ed82b5e320efb5beebed85ae8c #签名
18. Status: Downloaded newer image for mysql:latest
19. docker.io/library/mysql:latest #真实地址
20.
21. # 等价于
22. docker pull mysql
23. docker pull docker.io/library/mysql:latest
24.
25. #指定版本下载
```

docker rmi

docker rmi 删除镜像

```
1. [root@localhost docker]# docker rmi -f 镜像id #删除指定的镜像
2. [root@localhost docker]# docker rmi -f 镜像id 镜像id 镜像id #删除多个镜像
3. [root@localhost docker]# docker rmi -f $(docker images -aq) #删除全部的镜像
```

容器命令

说明：有了镜像才可以创建容器，拉取一个centos测试

```
1. docker pull centos
```

docker run

新建容器并启动 — docker run

1. **docker** run [可选参数] image

2.

3. *#参数说明*

4. --name="Name" 容器名字 tomcat01 , tomcat02 , 用来区分容器

5. -d 后台方式运行

6. -it 使用交互方式运行 , 进入容器查看内容

7. -p 指定容器端口 -p : 8080

8. -p ip:主机端口:容器端口

9. -p 主机端口:容器端口 (常用)

10. -p 容器端口

11. 容器端口

12. -P 随机指定端口

13.

14. *#测试 , 启动并进入容器*

15. [root@localhost **docker**]# **docker** run -it 300e315adb2f /bin/bash

16. [root@b39c31073d93 /]# ls *#查看容器内的centos , 基础版 , 命令不完善*

17. bin dev etc home lib lib64 lost+found media mnt opt proc **root** run sbin srv sys tmp usr **var**

18.

19. *#从容器中退回主机 exit*

20. [root@b39c31073d93 /]# **exit**

21. **exit**

22. [root@localhost **docker**]# ls

23. buildkit containers image network overlay2 plugins runtimes swarm tmp trust volumes

24. [root@localhost **docker**]#

docker ps

列出所有的运行的容器 — docker ps

1. *#docker ps命令*

2. *#列出当前正在运行的容器*

3. -a *#列出当前正在运行的容器+历史运行过的容器*

4. -h=? *#列出最近创建的容器*

5. -q *#只显示容器编号*

6. [root@localhost **docker**]# **docker ps**

7. CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

8. [root@localhost **docker**]# **docker ps -a**

9. CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

10. b39c31073d93 300e315adb2f **"/bin/bash"** 7 minutes ago **Exited (0)** 5 minutes ago beautiful_nightingale

11. 9b9a06222ffa bf756fb1ae65 **"/hello"** 3 hours ago **Exited (0)** 3 hours ago suspicious_mcnulty

退出容器

1. **exit** *#直接容器停止并退出*

2. **Ctrl+P+Q** *#容器不停止退出*

删除容器

1. **docker rm** 容器id *#删除指定容器 , 不能删除正在运行的 , 如果强制退出 rm -f*

2. **docker rm -f** \$(**docker ps -aq**) *#删除所有容器*

3. **docker ps -aq**|xargs **docker rm** *#删除所有容器*

启停容器

1. **docker start** 容器id *#启动容器*

2. **docker restart** 容器id *#重启容器*

3. **docker stop** 容器id *#停止当前正在运行的容器*

4. **docker kill** 容器id *#强制停止容器*

常用命令

后台启动容器

1. #命令 `docker run -d` 镜像名

2. `[root@localhost docker]# docker run -d centos`

3. `35da15ca7202429045677e3c460780881fc6ab0b87edfd0950880c1958ad345e`

4. `[root@localhost docker]# docker ps`

5.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

6.

7. #问题，`docker ps`一下，发现`centos`停止了

8.

9. #坑:`docker`容器使用后台运行，就必须要有个前台进程，`docker`发现没有应用，就会自动停止

10. #`nginx`，容器启动后，发现自己没有提供服务，就会立刻停止，就是没有程序了

查看日志

1. `docker logs -ft --tail number` 容器

2.

3. #自己编写一段shell脚本，使容器有运行

4. `[root@localhost docker]# docker run -d centos /bin/sh -c "while true;do echo ln1;sleep 1;done"`

5. `4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c`

6. `[root@localhost docker]# docker ps`

7.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

8.

4d293569fb0d	centos	"/bin/sh -c 'while t..."	6 seconds ago	Up 4 seconds		nervous_morse
--------------	--------	--------------------------	---------------	--------------	--	---------------

9. `[root@localhost docker]# docker logs -ft --tail 100 4d293569fb0d`

10. `2021-01-27T10:03:16.465680980Z ln1`

11. `2021-01-27T10:03:17.501246459Z ln1`

12. `2021-01-27T10:03:18.506364343Z ln1`

13. #-tf 显示日志，t是时间戳，f是逐条显示

14. #--tail number 要显示的日志条数

查看进程信息

1. # 命令 `docker top` 容器id

2. `[root@localhost docker]# docker top 4d293569fb0d`

3.

UID	PID	PPID	C	STIME	TTY
-----	-----	------	---	-------	-----

4.

root	51320	51299	0	10:03	?
00:00:00	/bin/sh -c while true;do echo ln1;sleep 1;done				

查看容器元数据



```
1. docker inspect 容器id
2. [root@localhost docker]# docker inspect 4d293569fb0d
3. [
4.     {
5.         "Id": "4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c",
6.         "Created": "2021-01-27T10:03:15.919253665Z",
7.         "Path": "/bin/sh",
8.         "Args": [
9.             "-c",
10.             "while true;do echo ln1;sleep 1;done"
11.         ],
12.         "State": {
13.             "Status": "running",
14.             "Running": true,
15.             "Paused": false,
16.             "Restarting": false,
17.             "OOMKilled": false,
18.             "Dead": false,
19.             "Pid": 51320,
20.             "ExitCode": 0,
21.             "Error": "",
22.             "StartedAt": "2021-01-27T10:03:16.46318534Z",
23.             "FinishedAt": "0001-01-01T00:00:00Z"
24.         },
25.         "Image": "sha256:300e315adb2f96afe5f0b2780b87f28ae95231fe3bdd1e16b9ba606307728f55",
26.         "ResolvConfPath":
27.             "/var/lib/docker/containers/4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c/resolv.conf",
28.         "HostnamePath":
29.             "/var/lib/docker/containers/4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c/hostname",
30.         "HostsPath":
31.             "/var/lib/docker/containers/4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c/hosts",
32.         "LogPath":
33.             "/var/lib/docker/containers/4d293569fb0d26f33722082c89da01be105cbb0d68b31a8a31ef001eeada0a8c/4d293569fb0d26f3372208
34.             json.log",
35.         "Name": "/nervous_morse",
36.         "RestartCount": 0,
37.         "Driver": "overlay2",
38.         "Platform": "linux",
39.         "MountLabel": "",
40.         "ProcessLabel": "",
41.         "AppArmorProfile": "",
42.         "ExecIDs": null,
43.         "HostConfig": {
44.             "Binds": null,
45.             "ContainerIDFile": "",
46.             "LogConfig": {
47.                 "Type": "json-file",
48.                 "Config": {}
49.             },
50.             "NetworkMode": "default",
51.             "PortBindings": {},
52.             "RestartPolicy": {
53.                 "Name": "no",
54.                 "MaximumRetryCount": 0
55.             },
56.             "AutoRemove": false,
57.             "VolumeDriver": "",
58.             "VolumesFrom": null,
59.             "CapAdd": null,
60.             "CapDrop": null,
61.             "CgroupnsMode": "host",
62.             "Dns": [],
63.             "DnsOptions": [],
64.             "DnsSearch": [],
65.             "ExtraHosts": null,
```

```
61.     "GroupAdd": null,
62.     "IpcMode": "private",
63.     "Cgroup": "",
64.     "Links": null,
65.     "OomScoreAdj": 0,
66.     "PidMode": "",
67.     "Privileged": false,
68.     "PublishAllPorts": false,
69.     "ReadonlyRootfs": false,
70.     "SecurityOpt": null,
71.     "UTSMode": "",
72.     "UsernsMode": "",
73.     "ShmSize": 67108864,
74.     "Runtime": "runc",
75.     "ConsoleSize": [
76.         0,
77.         0
78.     ],
79.     "Isolation": "",
80.     "CpuShares": 0,
81.     "Memory": 0,
82.     "NanoCpus": 0,
83.     "CgroupParent": "",
84.     "BlkioWeight": 0,
85.     "BlkioWeightDevice": [],
86.     "BlkioDeviceReadBps": null,
87.     "BlkioDeviceWriteBps": null,
88.     "BlkioDeviceReadIOps": null,
89.     "BlkioDeviceWriteIOps": null,
90.     "CpuPeriod": 0,
91.     "CpuQuota": 0,
92.     "CpuRealtimePeriod": 0,
93.     "CpuRealtimeRuntime": 0,
94.     "CpusetCpus": "",
95.     "CpusetMems": "",
96.     "Devices": [],
97.     "DeviceCgroupRules": null,
98.     "DeviceRequests": null,
99.     "KernelMemory": 0,
100.    "KernelMemoryTCP": 0,
101.    "MemoryReservation": 0,
102.    "MemorySwap": 0,
103.    "MemorySwappiness": null,
104.    "OomKillDisable": false,
105.    "PidsLimit": null,
106.    "Ulimits": null,
107.    "CpuCount": 0,
108.    "CpuPercent": 0,
109.    "IOMaximumIOps": 0,
110.    "IOMaximumBandwidth": 0,
111.    "MaskedPaths": [
112.        "/proc/asound",
113.        "/proc/acpi",
114.        "/proc/kcore",
115.        "/proc/keys",
116.        "/proc/latency_stats",
117.        "/proc/timer_list",
118.        "/proc/timer_stats",
119.        "/proc/sched_debug",
120.        "/proc/scsi",
121.        "/sys/firmware"
122.    ],
123.    "ReadonlyPaths": [
124.        "/proc/bus",
125.        "/proc/fs",
126.        "/proc/irq",
127.        "/proc/sys",
```

```
128.         "/proc/sysrq-trigger"
129.     ]
130. },
131.     "GraphDriver": {
132.         "Data": {
133.             "LowerDir":
134.             "/var/lib/docker/overlay2/576bafdfefbec84e113436cf29c4bd97c87a5b3e96bbfa9a7f207e8ddbc2b3304-
init/diff:/var/lib/docker/overlay2/3338d103a52bfc0ae4f9c97f7a9c677f25424bfa1472773745bbe67588c1daf4/diff",
135.             "MergedDir":
136.             "/var/lib/docker/overlay2/576bafdfefbec84e113436cf29c4bd97c87a5b3e96bbfa9a7f207e8ddbc2b3304/merged",
137.             "UpperDir":
138.             "/var/lib/docker/overlay2/576bafdfefbec84e113436cf29c4bd97c87a5b3e96bbfa9a7f207e8ddbc2b3304/diff",
139.             "WorkDir":
140.             "/var/lib/docker/overlay2/576bafdfefbec84e113436cf29c4bd97c87a5b3e96bbfa9a7f207e8ddbc2b3304/work"
141.         },
142.         "Name": "overlay2"
143.     },
144.     "Mounts": [],
145.     "Config": {
146.         "Hostname": "4d293569fb0d",
147.         "Domainname": "",
148.         "User": "",
149.         "AttachStdin": false,
150.         "AttachStdout": false,
151.         "AttachStderr": false,
152.         "Tty": false,
153.         "OpenStdin": false,
154.         "StdinOnce": false,
155.         "Env": [
156.             "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
157.         ],
158.         "Cmd": [
159.             "/bin/sh",
160.             "-c",
161.             "while true;do echo ln1;sleep 1;done"
162.         ],
163.         "Image": "centos",
164.         "Volumes": null,
165.         "WorkingDir": "",
166.         "Entrypoint": null,
167.         "OnBuild": null,
168.         "Labels": {
169.             "org.label-schema.build-date": "20201204",
170.             "org.label-schema.license": "GPLv2",
171.             "org.label-schema.name": "CentOS Base Image",
172.             "org.label-schema.schema-version": "1.0",
173.             "org.label-schema.vendor": "CentOS"
174.         }
175.     },
176.     "NetworkSettings": {
177.         "Bridge": "",
178.         "SandboxID": "b0a93bd3004afd04e40204879aa0f04d014a2667c29c29fdee364feffe36c5b8",
179.         "HairpinMode": false,
180.         "LinkLocalIPv6Address": "",
181.         "LinkLocalIPv6PrefixLen": 0,
182.         "Ports": {},
183.         "SandboxKey": "/var/run/docker/netns/b0a93bd3004a",
184.         "SecondaryIPAddresses": null,
185.         "SecondaryIPv6Addresses": null,
186.         "EndpointID": "11f2e2e59c8b2508946e897c1764fee4aea84c96fa9907fa32e9bedbbf90356d",
187.         "Gateway": "172.17.0.1",
188.         "GlobalIPv6Address": "",
189.         "GlobalIPv6PrefixLen": 0,
190.         "IPAddress": "172.17.0.2",
191.         "IPPrefixLen": 16,
192.         "IPv6Gateway": "",
193.         "MacAddress": "02:42:ac:11:00:02",
```

```
190.         "Networks": {
191.             "bridge": {
192.                 "IPAMConfig": null,
193.                 "Links": null,
194.                 "Aliases": null,
195.                 "NetworkID": "f09f41e23da993cacf550392e737b06e4ff0b8f44b35c98e7260e795fdb2c08e",
196.                 "EndpointID": "11f2e2e59c8b2508946e897c1764fee4aea84c96fa9907fa32e9bedbbf90356d",
197.                 "Gateway": "172.17.0.1",
198.                 "IPAddress": "172.17.0.2",
199.                 "IPPrefixLen": 16,
200.                 "IPv6Gateway": "",
201.                 "GlobalIPv6Address": "",
202.                 "GlobalIPv6PrefixLen": 0,
203.                 "MacAddress": "02:42:ac:11:00:02",
204.                 "DriverOpts": null
205.             }
206.         }
207.     }
208. }
209. ]
```

进入正在运行的容器

- 
1. *# 我们通常是用后台运行容器，需要进入容器，修改一些配置*
 2. `docker exec -it 容器id`
 3. `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`
 4. `[root@localhost docker]# docker exec -it 4d293569fb0d /bin/bash`
 5. `[root@4d293569fb0d /]# ls`
 6. `bin dev etc home lib lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var`
 7. `[root@4d293569fb0d /]# ps -ef`
 8.

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	10:03	?	00:00:00	/bin/sh -c while true;do echo ln1;sleep 1;done
root	923	0	0	10:18	pts/0	00:00:00	/bin/bash
root	983	1	0	10:19	?	00:00:00	/usr/bin/coreutils --coreutils-prog-shebang=sleep /usr/bin/sleep 1
root	984	923	0	10:19	pts/0	00:00:00	ps -ef
 9. root
 10. root
 11. root
 12. root
 - 13.
 14. *#方法二：*
 15. `docker attach 容器id`
 16. `[root@localhost docker]# docker attach 4d293569fb0d`
 17. 正在执行的当前代码
 - 18.
 19. *#docker exec #进入容器后开启一个新的终端，可以在里面操作（常用）*
 20. *#docker attach #进入容器正在执行的终端，不创建进程*

从容器拷贝文件到主机

1. `docker cp` 容器id : 容器内路径 主机路径

2. `[root@localhost home]# ls`

3. `docker`

4. `[root@localhost home]# docker ps`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0eb76c1ed9de	centos	"/bin/bash"	35 seconds ago	Up 34 seconds		dazzling_satoshi

7. #进入容器

8. `[root@localhost home]# docker attach 0eb76c1ed9de`

9. #在容器中创建一个文件

10. `[root@0eb76c1ed9de /]# cd /home`

11. `[root@0eb76c1ed9de home]# touch test.txt`

12. `[root@0eb76c1ed9de home]# ls`

13. `test.txt`

14. #退出容器并查看容器id

15. `[root@0eb76c1ed9de home]# exit`

16. `exit`

17. `[root@localhost home]# docker ps -a`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
0eb76c1ed9de	centos	"/bin/bash"	About a minute ago	Exited (0) 12 seconds ago	
4441f7a8fd7b	centos	"/bin/bash"	9 minutes ago	Exited (0) 9 minutes ago	
4d293569fb0d	centos	"/bin/sh -c 'while t...'"	31 minutes ago	Exited (137) 10 minutes ago	
35da15ca7202	centos	"/bin/bash"	38 minutes ago	Exited (0) 38 minutes ago	

23. #复制容器文件到主机

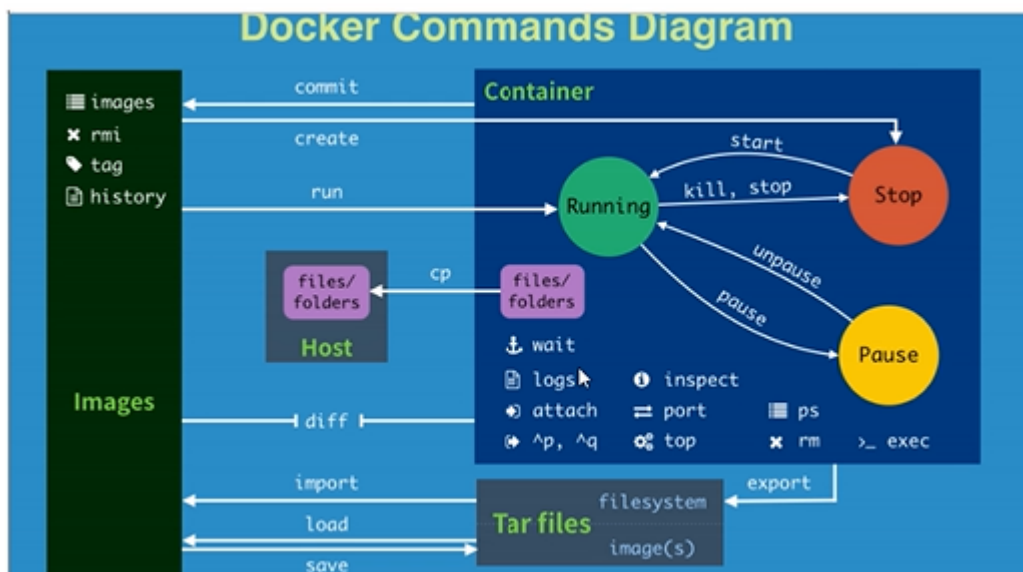
24. `[root@localhost home]# docker cp 0eb76c1ed9de:/home/test.txt /home`

25. `[root@localhost home]# ls`

26. `docker test.txt`

小结

1. attach	Attach to a running container	# 当前 <i>shell</i> 下 <i>attach</i> 连接指定运行镜像
2. build	Build an image from a Dockerfile	# 通过 <i>Dockerfile</i> 定制镜像
3. commit	Create a new image from a container changes	# 提交当前容器为新的镜像
4. cp 中	Copy files/folders from the containers filesystem to the host path	#从容器中拷贝 指定文件或者目录到宿主机
5. create	Create a new container	# 创建一个新的容器，同 <i>run</i> ，但不启动容器
6. diff	Inspect changes on a container's filesystem	# 查看 <i>docker</i> 容器变化
7. events	Get real time events from the server	# 从 <i>docker</i> 服务获取容器实时事件
8. exec	Run a command in an existing container	# 在已存在的容器上运行命令
9. export]	Stream the contents of a container as a tar archive	# 导出容器的内容流作为一个 <i>tar</i> 归档文件[对应 <i>import</i>
10. history	Show the history of an image	# 展示一个镜像形成历史
11. images	List images	# 列出系统当前镜像
12. import 应export]	Create a new filesystem image from the contents of a tarball	# 从tar包中的内容创建一个新的文件系统映像[对
13. info	Display system-wide information	# 显示系统相关信息
14. inspect	Return low-level information on a container	# 查看容器详细信息
15. kill	Kill a running container	# <i>kill</i> 指定 <i>docker</i> 容器
16. load	Load an image from a tar archive	# 从一个 <i>tar</i> 包中加载一个镜像[对应 <i>save</i>]
17. login	Register or Login to the docker registry server	# 注册或者登陆一个 <i>docker</i> 源服务器
18. logout	Log out from a Docker registry server	# 从当前 <i>Docker registry</i> 退出
19. logs	Fetch the logs of a container	# 输出当前容器日志信息
20. port	Lookup the public-facing port which is NAT-ed to PRIVATE_PORT	# 查看映射端口对应的容器内部源端口
21. pause	Pause all processes within a container	# 暂停容器
22. ps	List containers	# 列出容器列表
23. pull 像	Pull an image or a repository from the docker registry server	# 从docker镜像源服务器拉取指定镜像或者库镜
24. push	Push an image or a repository to the docker registry server	# 推送指定镜像或者库镜像至docker源服务器
25. restart	Restart a running container	# 重启运行的容器
26. rm	Remove one or more containers	# 移除一个或者多个容器
27. rmi 续或 -f 强制删除]	Remove one or more images	# 移除一个或多个镜像[无容器使用该镜像才可删除，否则需删除相关容器才可继
28. run	Run a command in a new container	# 创建一个新的容器并运行一个命令
29. save	Save an image to a tar archive	# 保存一个镜像为一个 <i>tar</i> 包[对应 <i>load</i>]
30. search	Search for an image on the Docker Hub	# 在 <i>docker hub</i> 中搜索镜像
31. start	Start a stopped containers	# 启动容器
32. stop	Stop a running containers	# 停止容器
33. tag	Tag an image into a repository	# 给源中镜像打标签
34. top	Lookup the running processes of a container	# 查看容器中运行的进程信息
35. unpause	Unpause a paused container	# 取消暂停容器
36. version	Show the docker version information	# 查看 <i>docker</i> 版本号
37. wait	Block until a container stops, then print its exit code	# 截取容器停止时的退出状态值



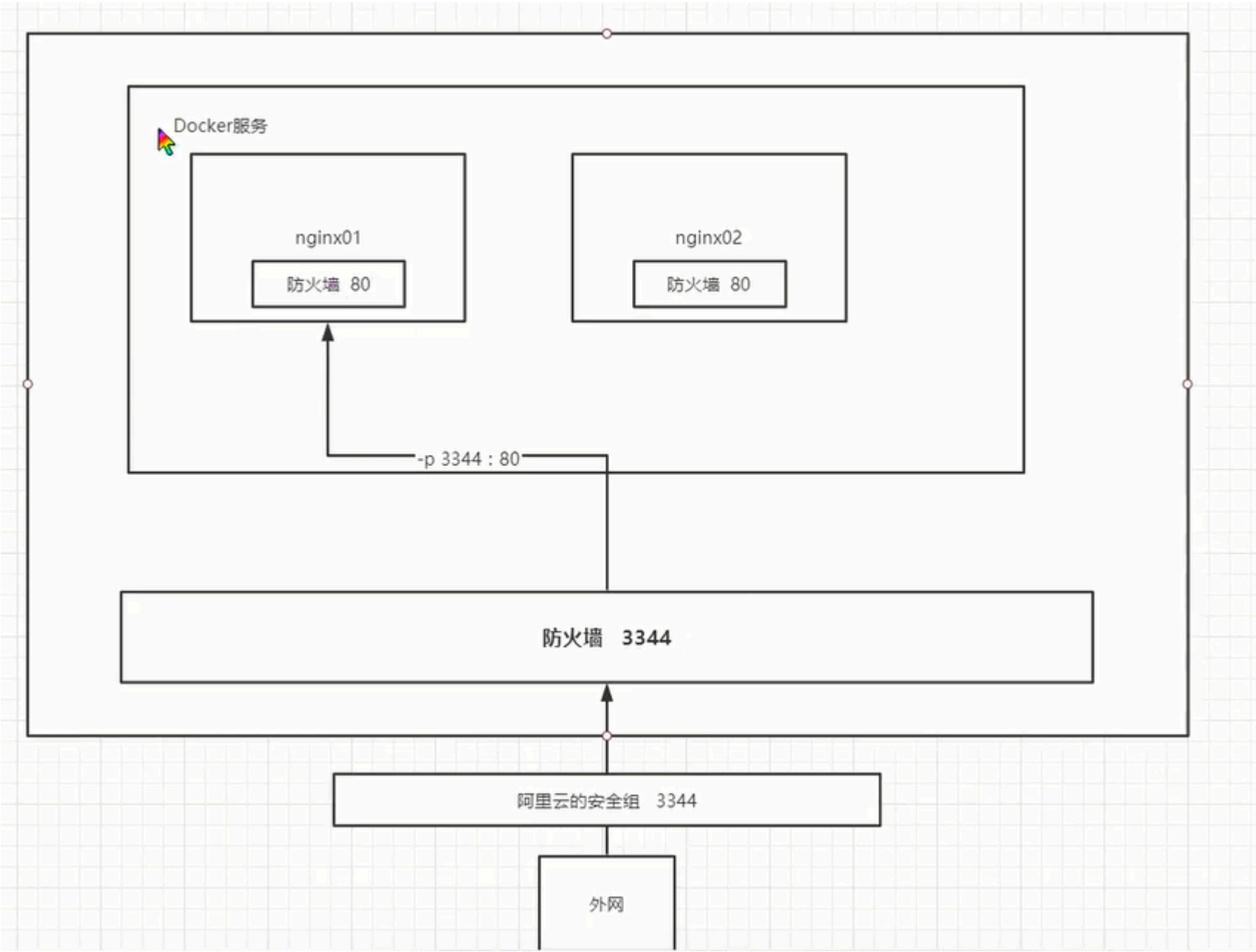
练习

1.安装nginx



```
1. # 1、搜索镜像 Search
2. # 2、下载镜像 Pull
3. # 3、运行测试
4. [root@localhost home]# docker images
5. REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
6. nginx            latest       f6d0b4767a6c  2 weeks ago  133MB
7. centos           latest       300e315adb2f  7 weeks ago  209MB
8.
9. #-d 后台运行
10. #--name 给容器命名
11. #-p 宿主机端口:容器内部端口      端口映射!
12. [root@localhost home]# docker run -d --name nginx01 -p 3344:80 nginx
13. 4c3ad394c42504adbe5c79ae84daf4970227f6f1dfcd9dee791f457f69f14e83
14. [root@localhost home]# docker ps
15. CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
16. 4c3ad394c425   nginx     "/docker-entrypoint. ...."  49 seconds ago Up 14 seconds  0.0.0.0:3344->80/tcp   nginx01
17. [root@localhost home]# curl localhost:3344
18. <!DOCTYPE html>
19. <html>
20. <head>
21. <title>Welcome to nginx!</title>
22. <style>
23.     body {
24.         width: 35em;
25.         margin: 0 auto;
26.         font-family: Tahoma, Verdana, Arial, sans-serif;
27.     }
28. </style>
29. </head>
30. <body>
31. <h1>Welcome to nginx!</h1>
32. <p>If you see this page, the nginx web server is successfully installed and
33. working. Further configuration is required.</p>
34.
35. <p>For online documentation and support please refer to
36. <a href="http://nginx.org/">nginx.org</a>.<br/>
37. Commercial support is available at
38. <a href="http://nginx.com/">nginx.com</a>.</p>
39.
40. <p><em>Thank you for using nginx.</em></p>
41. </body>
42. </html>
43.
44. #进入容器
45. [root@localhost home]# docker exec -it nginx01 /bin/bash
46. root@4c3ad394c425:/# whereis nginx
47. nginx: /usr/sbin/nginx /usr/lib/nginx /etc/nginx /usr/share/nginx
48. root@4c3ad394c425:/# cd /etc/nginx/
49. root@4c3ad394c425:/etc/nginx# ls
50. conf.d fastcgi_params koi-utf koi-win mime.types modules nginx.conf scgi_params uwsgi_params win-utf
```

暴露端口概念图

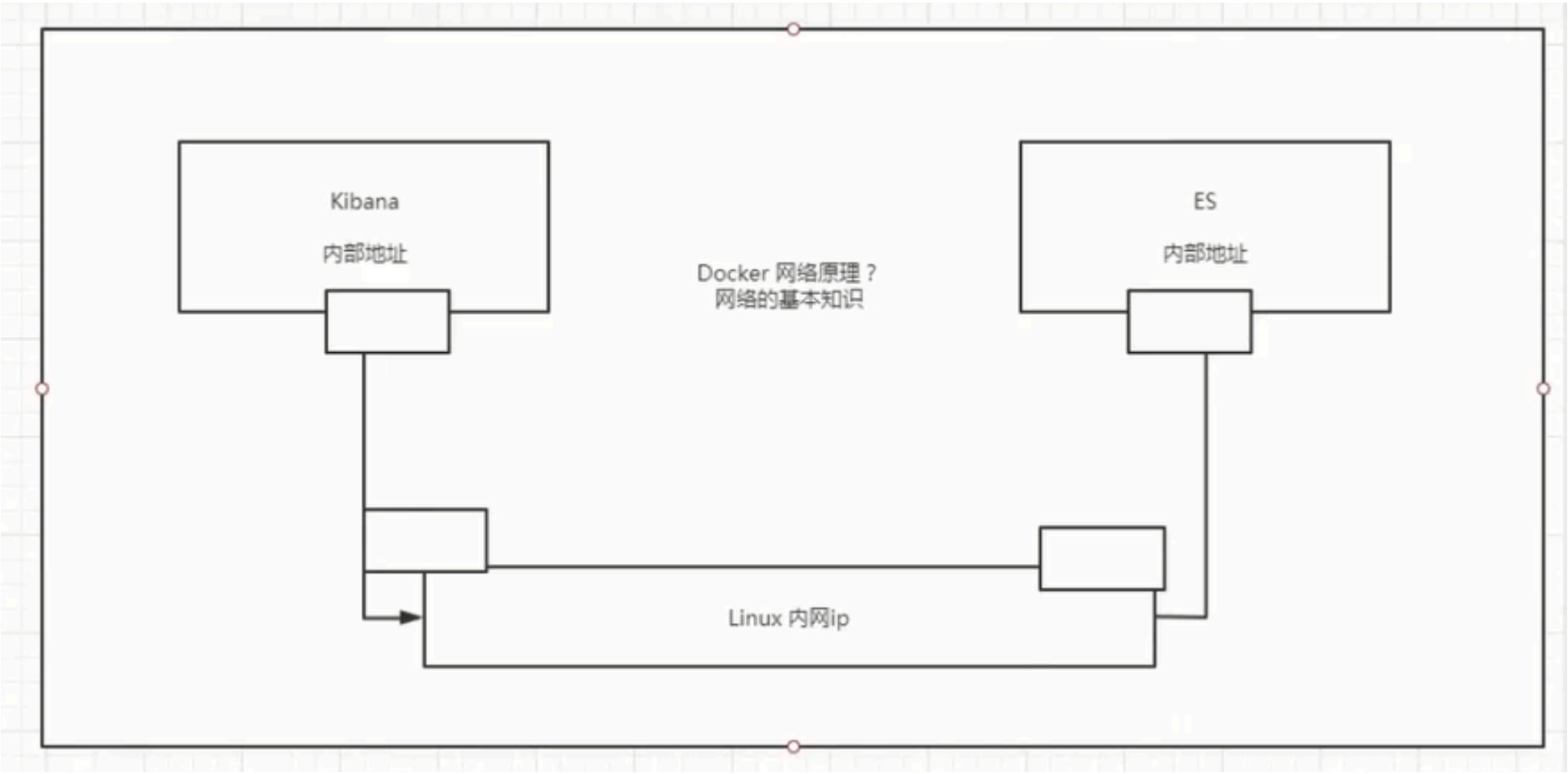


2.安装tomcat

1. # 官方的使用
2. `docker run -it --rm tomcat:9.0`
- 3.
4. #之前启动都是后台运行，停止了容器后，容器还可以查到，`docker run -it --rm` 一般用来测试，用完就删了
- 5.
6. # 下载再启动
7. `docker pull tomcat`
8. `docker run -d -p 3355:8080 --name toncat01 tomcat`
- 9.
10. #测试访问没问题
- 11.
12. #进入容器
- 13.
14. #发现问题：1、linux命令少了 2、没有webapps；是因为阿里云镜像默认是最小镜像，剔除不必要的东西，保证最小运行

3.部署es+kibanna

1. #es暴露端口很多，十分耗内存
2. #es数据一般放在安全目录，挂载
3. #--net somenetwork 网络配置
- 4.
5. #启动elasticsearch
6. `docker run -d --name elasticsearch -p 9200:9200 -p 9300:9300 -e "discovery.type=single-node" elasticsearch:7.6.2`
- 7.
8. #启动了linux就卡住，`docker stats` 看看cpu状态，发现es占用极大内存
9. #赶紧关闭容器，增加内存限制 Xms最小内存 Xmx最大内存
10. `docker run -d --name elasticsearch -p 9200:9200 -p 9300:9300 -e "discovery.type=single-node" -e ES_JAVA_OPTS="-Xms64m -Xmx512m" elasticsearch:7.6.2`



🔔 docker镜像讲解

🏆 镜像是什么？

镜像是一种轻量级的可执行的软件包，用来打包软件运行的环境和基于运行环境开发的软件，它包含软件运行所需的所有内容，包括代码、依赖库、环境变量、配置文件。

🏆 联合文件系统UnionF

UnionFS是一种分层、轻量级、高性能的文件系统，它支持对文件系统的修改作为一次提交来一层层叠加，同时可以将不同目录挂载到统一虚拟文件系统下。

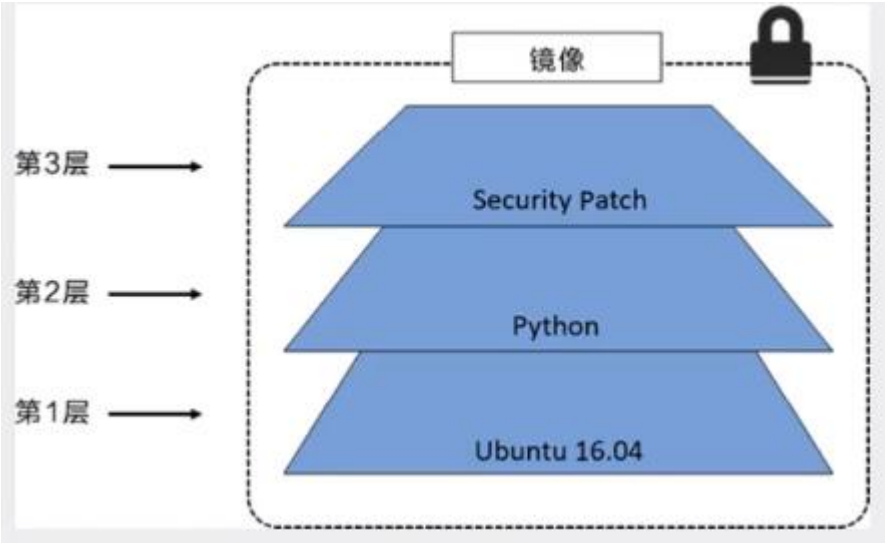
Union文件系统是Docker镜像的基础，镜像可以通过分层来继承，基于基础镜像（没有父镜像），可以制作各种具体的应用镜像。

特性：一次同时加载多个文件系统，但从外面看起来，只能看到一个文件系统，联合加载会把各层文件系统叠加起来，这样最终的文件系统会包含所有底层的文件和目录。

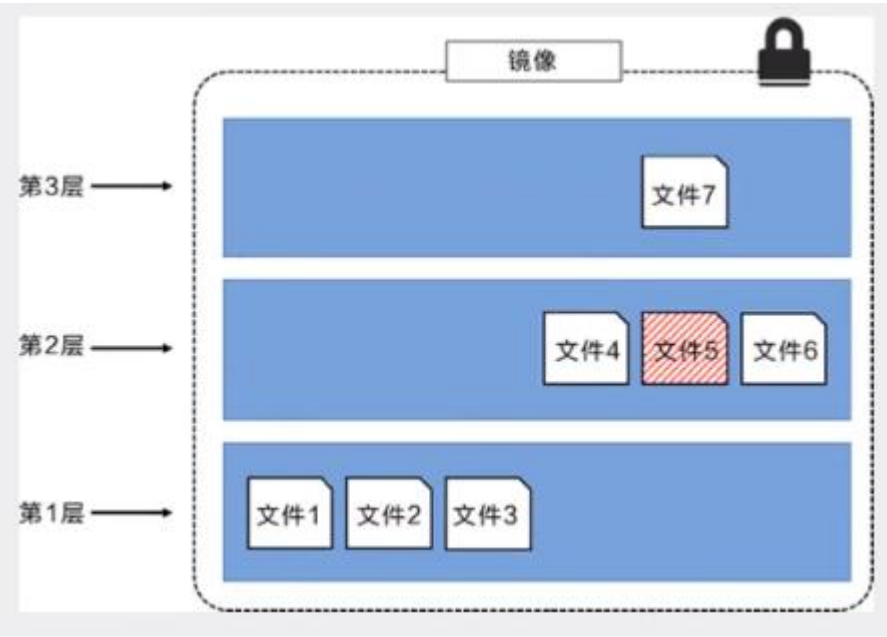
🏆 分层原理

每次pull命令下载镜像时，都可以看到是分层下载的，比如一个redis7包含7层（Layer），当下载redis8的时候可能只需要下载最新的三层，redis7**用的层不需要下载。

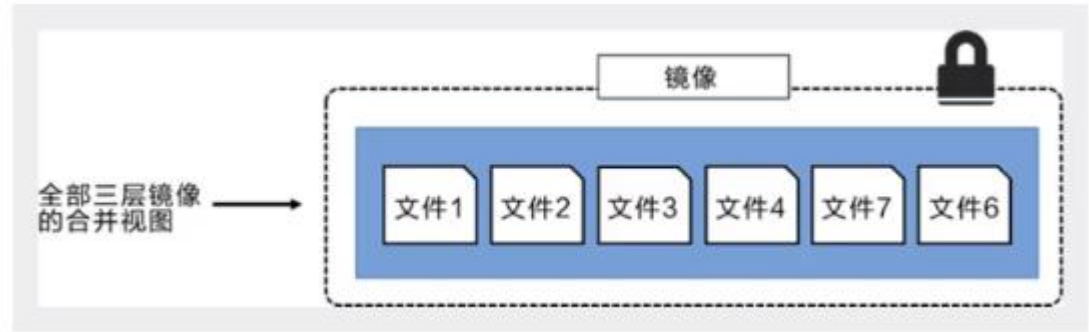
再比如有多个镜像都从相同的 Base 镜像构建而来，那么宿主机只需在磁盘上保留一份 base 镜像，同时内存中也只需要加载一份 base 镜像，这样就可以为所有的容器服务了，而且镜像的每一层都可以被共享。



docker中的所有镜像也是分层的，起始于一个基础镜像层，当修改或增加都会在这之上创建新的镜像层，类似添加安全补丁。

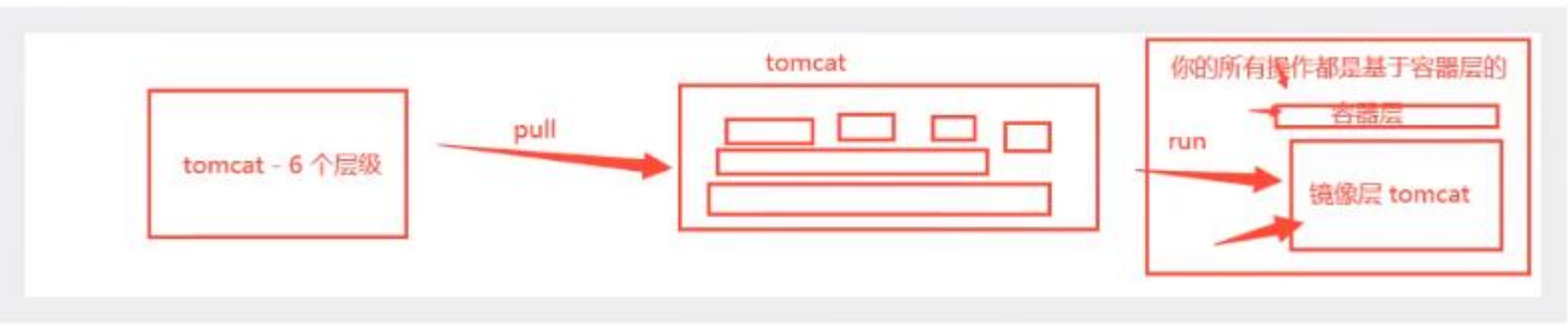


这种情况下，上层镜像层中的文件覆盖了底层镜像层中的文件。这样就使得文件的更新版本作为一个新镜像层添加到镜像当中。



docker镜像都是只读的，当容器启动时，一个新的可写层被加载到镜像的顶部，这一层被称为容器层，容器层下的是镜像层

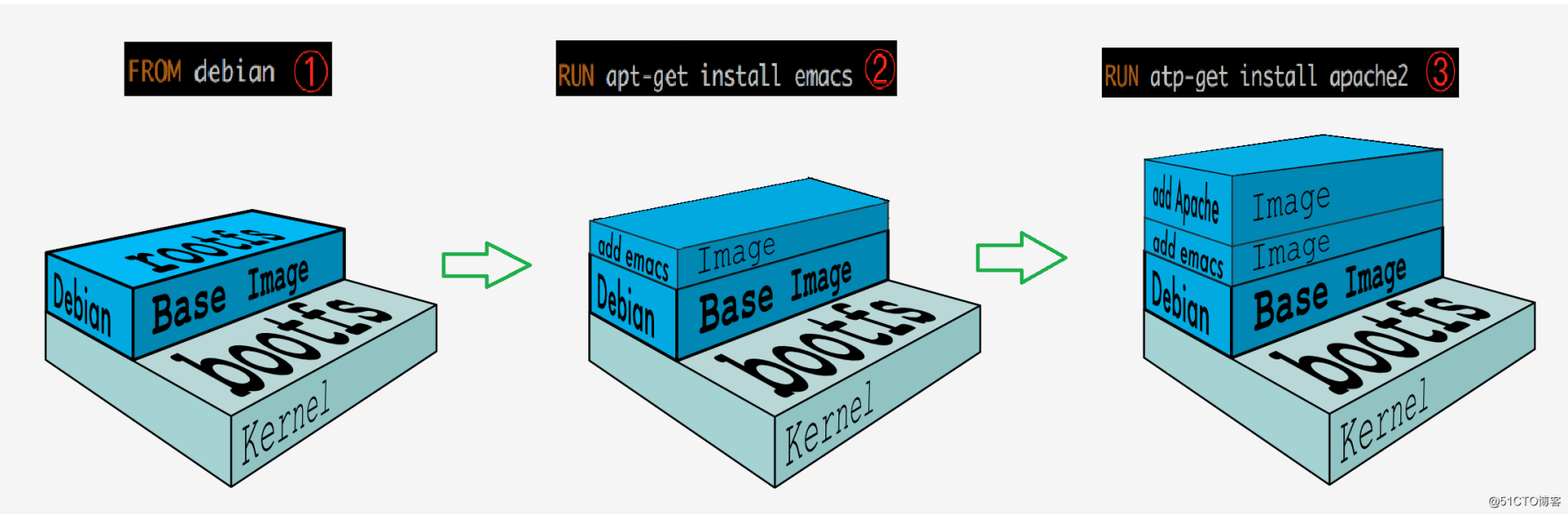
Docker 通过存储引擎（新版本采用快照机制）的方式来实现镜像层堆栈，并保证多镜像层对外展示为统一的文件系统。



🏆 镜像加载原理

bootfs(file system) 主要包含bootloader和kernel,boot加载器主要是引导加载内核，linux刚启动时会加载bootfs文件系统，在Docker镜像的最底层是bootfs。这一层与我们典型Linux/Unix系统是一样的，包含boot加载器和内核。当boot加载完成之后整个内核就在内存中了，此时内存使用权已有bootfs转交给内核，此时系统会卸载bootfs。

rootfs(root file system)是bootfs之上，包含的就是典型的Linux系统中的/dev,/proc,/bin,/etc等标准目录和文件。
rootfs就是各种不同操作系统发行版，比如Ubuntu,Centos等等。对于Linux的不同发行版，bootfs基本都是一样的，rootfs会有差别，因此不同发行版可以公用bootfs。



虚拟机启动分钟级别，容器是秒级启动的原因就在这

🏆 commit镜像

1. `docker commit` 提交容器，类似于vm的快照
- 2.
3. `docker commit -m="提交的描述信息" -a="作者" 容器id 目标镜像名:[TAG]`

1. #1、启动一个默认的tomcat
2. #2、发现这个是默认的，无webapps应用，官方镜像是默认没文件的
3. #3、拷贝进去webapps文件
4. #4、通过commit提交一个镜像
5. [root@localhost home]# `docker ps`
6.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
add4253c07c8	040bdb29ab37	"/bin/bash"	8 minutes ago	Up 8 minutes	8080/tcp	interesting_benz
8. [root@localhost home]# `docker commit -m="ln1" -a="cola" add4253c07c8 tomcat02:1.0`
9. [root@localhost home]# `docker images`
10.

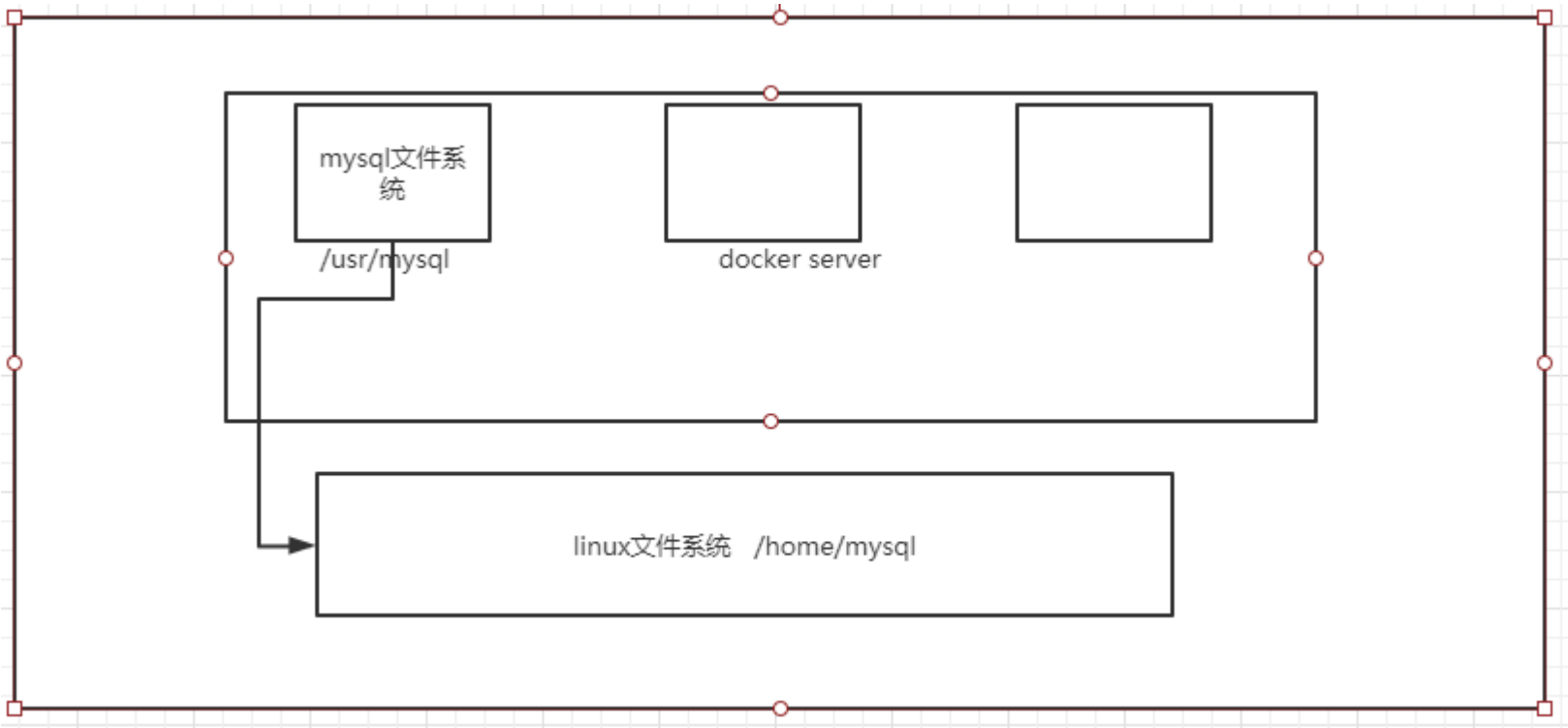
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
tomcat02	1.0	2fb96bb6ec85	47 seconds ago	653MB
tomcat	9.0	040bdb29ab37	2 weeks ago	649MB
- 12.

🔔 容器数据卷

🏆 什么是容器数据卷？

容器之间可以有一个数据共享的技术，将Docker容器中产生的数据同步到本地。
这就是卷技术：目录的挂载，将我们容器内的目录，挂载到linux上面，实现双向数据同步。

总结：容器的持久化操作。容器内也是可以数据共享的！



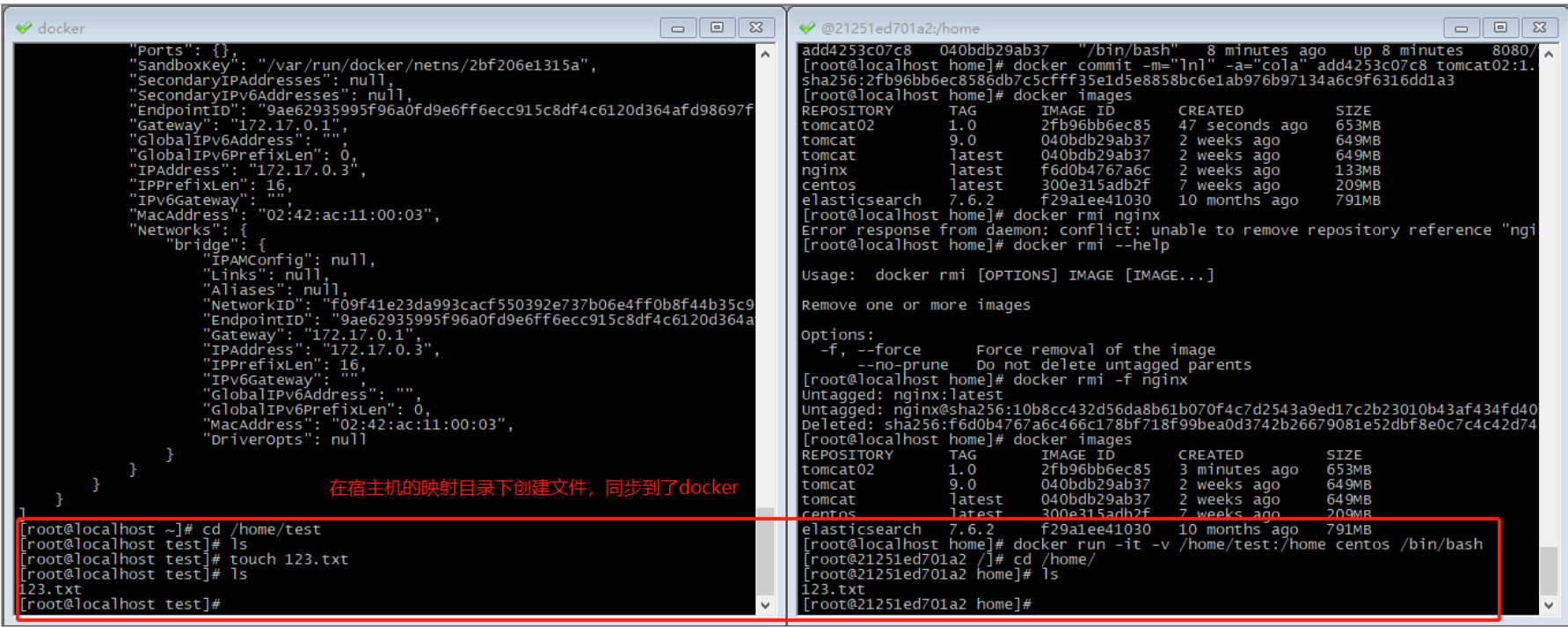
🏆 使用数据卷

方式一：直接使用命令挂载 -v

1. `docker run -it -v 主机目录:容器内目录`
- 2.
3. #测试，将docker容器的home目录挂载到宿主机/home/test
4. [root@localhost home]# `docker run -it -v /home/test:/home centos /bin/bash`
5. #启动起来后可以通'`docker inspect 容器id`'查看具体信息



测试文件同步



再次测试

- 1、停止容器
- 2、宿主机修改文件
- 3、启动容器
- 4、容器内数据依旧会同步

结论：我们只需在本地修改即可，容器会自动同步

🏆 练习：安装mysql

思考：mysql的数据持久化问题

1. #获取镜像
2. [root@localhost home]# docker pull mysql:5.7
3. #运行容器，需要做数据挂载！启动mysql，是需要配置密码的！！！
4. #官方测试:docker run --name some-mysql -e MYSQL_PASSWORD=mypassword -d msql:tag
5.
6. #自己启动的
7. -d 后台运行
8. -p 端口映射
9. -v 卷挂载
10. --name 容器名字
11. [root@localhost home]# docker images
12. REPOSITORY TAG IMAGE ID CREATED SIZE
13. mysql 5.7 a70d36bc331a 9 days ago 449MB
14. [root@localhost home]# docker run -d -p 3310:3306 -v /home/mysql/conf:/etc/mysql/conf.d -v /home/mysql/data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123456 --name mysql01 a70d36bc331a
15.
16. #启动之后，在本地测试
17. #使用工具连接到服务器的3310，可以连接 注:主机3310和容器3306做了映射

测试：将容器删除，发现挂载到本地的数据卷没有丢失，实现了容器数据持久化的功能

🏆 具名和匿名挂载

📄

```
1. # 匿名挂载
2. -v 容器内路径
3. docker run -d -P --name nginx01 -v /etc/nginx nginx
4.
5. # 查看所有volume的情况
6. [root@localhost home]# docker volume ls
7. DRIVER      VOLUME NAME
8. local       d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590
9.
10. #这里发现，这种就说匿名挂载，我们在-v只写了容器内路径，没写容器外路径
11.
12. #具名挂载
13. -v 卷名:容器内地址
14. [root@localhost home]# docker run -d -P --name nginx02 -v juming-nginx:/etc/nginx f6d0b4767a6c
15. 00d100907bb5c6c6275f807225c628b693c12d1a4052c597159bdef588b00aee
16. [root@localhost home]# docker volume ls
17. DRIVER      VOLUME NAME
18. local       d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590
19. local       juming-nginx
20.
21. [root@localhost home]# docker inspect volume juming-nginx
22. [
23.     {
24.         "CreatedAt": "2021-01-29T09:02:24Z",
25.         "Driver": "local",
26.         "Labels": null,
27.         "Mountpoint": "/var/lib/docker/volumes/juming-nginx/_data",
28.         "Name": "juming-nginx",
29.         "Options": null,
30.         "Scope": "local"
31.     }
32. ]
33.
34. #查看一下这个卷：
35. [root@localhost home]# docker volume ls
36. DRIVER      VOLUME NAME
37. local       d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590
38. [root@localhost home]# docker volume inspect d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590
39. [
40.     {
41.         "CreatedAt": "2021-01-28T17:31:48Z",
42.         "Driver": "local",
43.         "Labels": null,
44.         "Mountpoint":
45.         "/var/lib/docker/volumes/d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590/_data",
46.         "Name": "d07042b1008f32333efb224edd204cdf4d0bbbf29bd9ea017721a9ba76244590",
47.         "Options": null,
48.         "Scope": "local"
49.     }
50. ]
```

所有docker容器内的卷，在没有指定目录的情况下都是在==/var/lib/docker/volume/xxxx/data==

我们可以通过具名挂载可以方便的找到我们的一个卷，大多数情况使用==具名挂载==

- 📄
- 1. # 如何确定是具名挂载还是匿名挂载，还是指定路径
 - 2. -v 容器内路径 #匿名挂载
 - 3. -v 卷名:容器内路径 #具名挂载
 - 4. -v /主机路径:/容器内路径 #指定路径挂载

拓展

- 
1. *#通过-v 容器内路径:ro rw 来改变读写权限*
 2. ro **readonly** *#只读*
 3. rw **readwrite** *#可读 可写*
 - 4.
 5. *#一旦这个设置了容器权限，容器对我们挂载出来的内容就有限定了*
 6. **docker** run -d -P --name nginx01 -v juming-nginx:/etc/nginx:ro nginx
 7. **docker** run -d -P --name nginx01 -v juming-nginx:/etc/nginx:rw nginx
 - 8.
 9. *#ro 只要看到ro就说明这个路径只能通过宿主机来操作，容器内部是无法操作的*

初识Dockerfile

dockerfile就说用来构建docker镜像的构建文件！ 命令脚本

通过这个脚本可以生成镜像，镜像是一层一层的，脚本一个个的命令，每个命令都是一层



```
1. #创建一个dockerfile文件，名字可以随机，建议dockerfile
2. #文件中的内容 指令（大写）参数
3. 示例：
4. FROM centos
5. VOLUME ["volume01","volume02"]
6. CMD echo "-----end-----"
7. CMD /bin/bash
8.
9. #通过构建镜像时直接挂载volume01和volume02
10. #这里的每个命令，就是镜像的一层；注意：命令后面有个点(.)!!!
11. [root@localhost docker-test-volume]# docker build -f /home/docker-test-volume/dockerfile1 -t ln1/300e315adb2f .
12. Sending build context to Docker daemon 2.048kB
13. Step 1/4 : FROM centos
14. ----> 300e315adb2f
15. Step 2/4 : VOLUME ["volume01","volume02"]
16. ----> Running in 50a9f53caf8f
17. Removing intermediate container 50a9f53caf8f
18. ----> 13267a46ad39
19. Step 3/4 : CMD echo "-----end-----"
20. ----> Running in 1f2a0c70fb8b
21. Removing intermediate container 1f2a0c70fb8b
22. ----> a374c3509d39
23. Step 4/4 : CMD /bin/bash
24. ----> Running in f6cf70357097
25. Removing intermediate container f6cf70357097
26. ----> f4ec5b7c9126
27. Successfully built f4ec5b7c9126
28. Successfully tagged ln1/300e315adb2f:latest
29.
30. #启动自己写的容器
31. [root@localhost docker-test-volume]# docker run -it f4ec5b7c9126 /bin/bash
32. [root@d2b6cb9f4d6c /]# ls -l
33. total 0
34. lrwxrwxrwx. 1 root root 7 Nov 3 15:22 bin -> usr/bin
35. drwxr-xr-x. 5 root root 360 Jan 29 10:11 dev
36. drwxr-xr-x. 1 root root 66 Jan 29 10:11 etc
37. drwxr-xr-x. 2 root root 6 Nov 3 15:22 home
38. lrwxrwxrwx. 1 root root 7 Nov 3 15:22 lib -> usr/lib
39. lrwxrwxrwx. 1 root root 9 Nov 3 15:22 lib64 -> usr/lib64
40. drwx----- 2 root root 6 Dec 4 17:37 lost+found
41. drwxr-xr-x. 2 root root 6 Nov 3 15:22 media
42. drwxr-xr-x. 2 root root 6 Nov 3 15:22 mnt
43. drwxr-xr-x. 2 root root 6 Nov 3 15:22 opt
44. dr-xr-xr-x. 123 root root 0 Jan 29 10:11 proc
45. dr-xr-x--- 2 root root 162 Dec 4 17:37 root
46. drwxr-xr-x. 11 root root 163 Dec 4 17:37 run
47. lrwxrwxrwx. 1 root root 8 Nov 3 15:22 sbin -> usr/sbin
48. drwxr-xr-x. 2 root root 6 Nov 3 15:22 srv
49. dr-xr-xr-x. 13 root root 0 Jan 26 09:40 sys
50. drwxrwxrwt. 7 root root 145 Dec 4 17:37 tmp
51. drwxr-xr-x. 12 root root 144 Dec 4 17:37 usr
52. drwxr-xr-x. 20 root root 262 Dec 4 17:37 var
53. drwxr-xr-x. 2 root root 6 Jan 29 10:11 volume01
54. drwxr-xr-x. 2 root root 6 Jan 29 10:11 volume02
55. 发现有volume01和volume02
```

```
[root@localhost docker-test-volume]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
lnl/300e315adb2f    latest             f4ec5b7c9126       5 minutes ago      209MB
tomcat02            1.0                2fb96bb6ec85       20 hours ago       653MB
mysql               5.7                a70d36bc331a       10 days ago        449MB
tomcat              9.0                040bdb29ab37       2 weeks ago        649MB
tomcat              latest             040bdb29ab37       2 weeks ago        649MB
nginx               latest             f6d0b4767a6c       2 weeks ago        133MB
centos              latest             300e315adb2f       7 weeks ago        209MB
elasticsearch       7.6.2             f29a1ee41030       10 months ago      791MB
[root@localhost docker-test-volume]# docker run -it f4ec5b7c9126 /bin/bash
[root@d2b6cb9f4d6c /]# ls -l
total 0
lrwxrwxrwx.    1 root root    7 Nov  3 15:22 bin -> usr/bin
drwxr-xr-x.    5 root root  360 Jan 29 10:11 dev
drwxr-xr-x.    1 root root    66 Jan 29 10:11 etc
drwxr-xr-x.    2 root root    6 Nov  3 15:22 home
lrwxrwxrwx.    1 root root    7 Nov  3 15:22 lib -> usr/lib
lrwxrwxrwx.    1 root root    9 Nov  3 15:22 lib64 -> usr/lib64
drwx-----.   2 root root    6 Dec  4 17:37 lost+found
drwxr-xr-x.    2 root root    6 Nov  3 15:22 media
drwxr-xr-x.    2 root root    6 Nov  3 15:22 mnt
drwxr-xr-x.    2 root root    6 Nov  3 15:22 opt
dr-xr-xr-x.   123 root root    0 Jan 29 10:11 proc
dr-xr-xr-x.    2 root root   162 Dec  4 17:37 root
drwxr-xr-x.   11 root root   163 Dec  4 17:37 run
lrwxrwxrwx.    1 root root    8 Nov  3 15:22 sbin -> usr/sbin
drwxr-xr-x.    2 root root    6 Nov  3 15:22 srv
dr-xr-xr-x.   13 root root    0 Jan 26 09:40 sys
drwxrwxrwt.    7 root root   145 Dec  4 17:37 tmp
drwxr-xr-x.   12 root root   144 Dec  4 17:37 usr
drwxr-xr-x.   20 root root   262 Dec  4 17:37 var
drwxr-xr-x.    2 root root    6 Jan 29 10:11 volume01
drwxr-xr-x.    2 root root    6 Jan 29 10:11 volume02
[root@d2b6cb9f4d6c /]#
```

此目录即生成镜像的时候自动挂载的，数据卷目录

1. 查看一下卷挂载的路径
2. [root@localhost docker-test-volume]# docker ps
- | CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS | PORTS |
|-----------------|--------------|-------------------------|---------------|--------------|-----------------------|
| d2b6cb9f4d6c | f4ec5b7c9126 | "/bin/bash" | 3 minutes ago | Up 3 minutes | |
| amazing_mclaren | | | | | |
| aa03b92e7ed5 | f6d0b4767a6c | "/docker-entrypoint..." | 17 hours ago | Up 17 hours | 0.0.0.0:49153->80/tcp |
| nginx01 | | | | | |
6. [root@localhost docker-test-volume]# docker inspect d2b6cb9f4d6c
7. "Mounts": [
8. {
9. "Type": "volume",
10. "Name": "eadc15969bac9f58eb106fa49dfbd97bf441cf4b0d3c31edef81f5c2fd3fae79",
11. "Source":
12. "/var/lib/docker/volumes/eadc15969bac9f58eb106fa49dfbd97bf441cf4b0d3c31edef81f5c2fd3fae79/_data",
13. "Destination": "volume01",
14. "Driver": "local",
15. "Mode": "",
16. "RW": true,
17. "Propagation": ""
18. },
19. {
20. "Type": "volume",
21. "Name": "13201de1d58f58f0405c6c4065cc8ea641e40452da765baaa4533a372f61f65d",
22. "Source":
23. "/var/lib/docker/volumes/13201de1d58f58f0405c6c4065cc8ea641e40452da765baaa4533a372f61f65d/_data",
24. "Destination": "volume02",
25. "Driver": "local",
26. "Mode": "",
27. "RW": true,
28. "Propagation": ""
29. }
30.],

🏆 数据卷容器

多个mysql同步数据



启动多个容器，通过刚才写的镜像启动

```
[root@localhost docker-test-volume]# docker run -it --name docker01 ln1/300e315adb2f
[root@3a13242c10f5 /]# ls -l
total 0
lrwxrwxrwx. 1 root root 7 Nov 3 15:22 bin -> usr/bin
drwxr-xr-x. 5 root root 360 Jan 29 10:34 dev
drwxr-xr-x. 1 root root 66 Jan 29 10:33 etc
drwxr-xr-x. 2 root root 6 Nov 3 15:22 home
lrwxrwxrwx. 1 root root 7 Nov 3 15:22 lib -> usr/lib
lrwxrwxrwx. 1 root root 9 Nov 3 15:22 lib64 -> usr/lib64
drwx----- 2 root root 6 Dec 4 17:37 lost+found
drwxr-xr-x. 2 root root 6 Nov 3 15:22 media
drwxr-xr-x. 2 root root 6 Nov 3 15:22 mnt
drwxr-xr-x. 2 root root 6 Nov 3 15:22 opt
dr-xr-xr-x. 119 root root 0 Jan 29 10:33 proc
dr-xr-x--- 2 root root 162 Dec 4 17:37 root
drwxr-xr-x. 11 root root 163 Dec 4 17:37 run
lrwxrwxrwx. 1 root root 8 Nov 3 15:22 sbin -> usr/sbin
drwxr-xr-x. 2 root root 6 Nov 3 15:22 srv
dr-xr-xr-x. 13 root root 0 Jan 26 09:40 sys
drwxrwxrwt. 7 root root 145 Dec 4 17:37 tmp
drwxr-xr-x. 12 root root 144 Dec 4 17:37 usr
drwxr-xr-x. 20 root root 262 Dec 4 17:37 var
drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume01
drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume02
```

1. [root@localhost docker-test-volume]# docker run -it --name docker01 ln1/300e315adb2f
2. [root@localhost docker-test-volume]# docker run -it --name docker02 --volumes-from docker01 ln1/300e315adb2f
3. [root@b06c09d1920f /]# ls -l
4. total 0
5. lrwxrwxrwx. 1 root root 7 Nov 3 15:22 bin -> usr/bin
6. drwxr-xr-x. 5 root root 360 Jan 29 10:36 dev
7. drwxr-xr-x. 1 root root 66 Jan 29 10:36 etc
8. drwxr-xr-x. 2 root root 6 Nov 3 15:22 home
9. lrwxrwxrwx. 1 root root 7 Nov 3 15:22 lib -> usr/lib
10. lrwxrwxrwx. 1 root root 9 Nov 3 15:22 lib64 -> usr/lib64
11. drwx----- 2 root root 6 Dec 4 17:37 lost+found
12. drwxr-xr-x. 2 root root 6 Nov 3 15:22 media
13. drwxr-xr-x. 2 root root 6 Nov 3 15:22 mnt
14. drwxr-xr-x. 2 root root 6 Nov 3 15:22 opt
15. dr-xr-xr-x. 120 root root 0 Jan 29 10:36 proc
16. dr-xr-x--- 2 root root 162 Dec 4 17:37 root
17. drwxr-xr-x. 11 root root 163 Dec 4 17:37 run
18. lrwxrwxrwx. 1 root root 8 Nov 3 15:22 sbin -> usr/sbin
19. drwxr-xr-x. 2 root root 6 Nov 3 15:22 srv
20. dr-xr-xr-x. 13 root root 0 Jan 26 09:40 sys
21. drwxrwxrwt. 7 root root 145 Dec 4 17:37 tmp
22. drwxr-xr-x. 12 root root 144 Dec 4 17:37 usr
23. drwxr-xr-x. 20 root root 262 Dec 4 17:37 var
24. drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume01
25. drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume02

```
[root@localhost docker-test-volume]# docker run -it --name docker02 --volumes-from docker01 1n1/300e315adb2f
[root@b06c09d1920f /]# ls -l
total 0
lrwxrwxrwx. 1 root root 7 Nov 3 15:22 bin -> usr/bin
drwxr-xr-x. 5 root root 360 Jan 29 10:36 dev
drwxr-xr-x. 1 root root 66 Jan 29 10:36 etc
drwxr-xr-x. 2 root root 6 Nov 3 15:22 home
lrwxrwxrwx. 1 root root 7 Nov 3 15:22 lib -> usr/lib
lrwxrwxrwx. 1 root root 9 Nov 3 15:22 lib64 -> usr/lib64
drwx----- 2 root root 6 Dec 4 17:37 lost+found
drwxr-xr-x. 2 root root 6 Nov 3 15:22 media
drwxr-xr-x. 2 root root 6 Nov 3 15:22 mnt
drwxr-xr-x. 2 root root 6 Nov 3 15:22 opt
dr-xr-xr-x. 120 root root 0 Jan 29 10:36 proc
dr-xr-xr-x. 2 root root 162 Dec 4 17:37 root
drwxr-xr-x. 11 root root 163 Dec 4 17:37 run
lrwxrwxrwx. 1 root root 8 Nov 3 15:22 sbin -> usr/sbin
drwxr-xr-x. 2 root root 6 Nov 3 15:22 srv
dr-xr-xr-x. 13 root root 0 Jan 26 09:40 sys
drwxrwxrwt. 7 root root 145 Dec 4 17:37 tmp
drwxr-xr-x. 12 root root 144 Dec 4 17:37 usr
drwxr-xr-x. 20 root root 262 Dec 4 17:37 var
drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume01
drwxr-xr-x. 2 root root 6 Jan 29 10:33 volume02
```

--volumes-from相当于继承了docker01

docker01创建的内容同步到了docekr02

多个mysql实现数据共享

1. [root@localhost home]# docker run -d -p 3310:3306 -v /etc/mysql/conf.d -v /var/lib/mysql -e MYSQL_ROOT_PASSWORD=123456 --name mysql01 mysql:5.7
2. [root@localhost home]# docker run -d -p 3310:3306 -e MYSQL_ROOT_PASSWORD=123456 --name mysql02 --volumes-from mysql01 mysql:5.7
- 3.
4. #此时，可以实现两个容器的数据同步

🏆 结论

容器之间配置信息的传递，数据卷容器的生命周期一直持续到没有容器使用为止

但是一旦持久化到了本地，这个时候本地的数据是不会删除的

🔔 DockerFile

🏆 dockerfile介绍

dockerfile 是用来构建docker镜像的文件！ 命令行参数脚本

构建步骤：

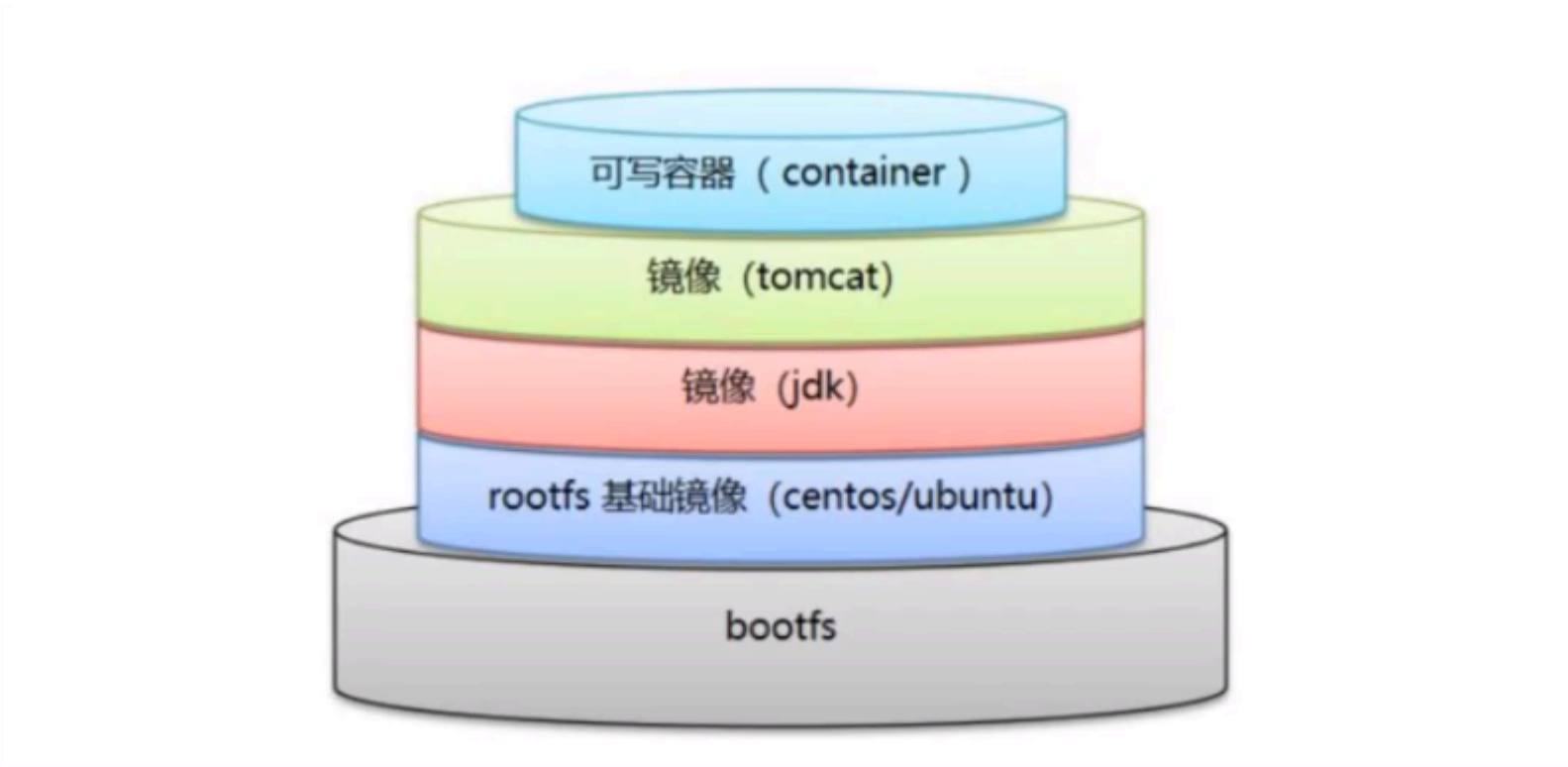
- 1.编写一个dockerfile文件
- 2.docker build 构建成为一个镜像
- 3.docker run 运行镜像
- 4.docker push 发布镜像

1. FROM scratch
2. ADD centos-7-x86_64-docker.tar.xz /
- 3.
4. LABEL \
5. org.label-schema.schema-version = " 1.0" \
6. org.label-schema.name = " CentOS Base Image" \
7. org.label-schema.vendor = " CentOS" \
8. org.label-schema.license = " GPLv2" \
9. org.label-schema.build-date = " 20201113" \
10. org.opencontainers.image.title = " CentOS Base Image" \
11. org.opencontainers.image.vendor = " CentOS" \
12. org.opencontainers.image.licenses = "GPL-2.0" \
13. org.opencontainers.image.created = " 2020-11-13 00:00:00 + 00:00"
- 14.
15. CMD ["/ bin / bash"]

🏆 dockerfile 构建

基础知识：

- 1.每个保留关键字（指令）都必须是大写字母
- 2.执行从上到下顺序
- 3.#表示注释
- 4.每个指令都会创建提交一层新镜像并提交



dockerfile：构建文件，定义了一切的步骤，源代码

dockerimage：通过dockerfile构建生成的镜像，最终发布和运行的产品

docker容器：容器就是镜像运行起来提供服务

🏆 dockerfile指令

- | | |
|---------------|--|
| 1. FROM | # 基础镜像层，一切从这里开始构建 |
| 2. MAINTAINER | # 镜像作者，姓名+邮箱 |
| 3. RUN | # 镜像构建的时候需要运行的命令 |
| 4. ADD | # 步骤，如果构建tomcat镜像，放入tomcat压缩包！添加内容 |
| 5. WORKDIR | # 镜像的工作目录 |
| 6. VOLUME | # 挂载的目录 |
| 7. EXPOSE | # 暴露配置端口 |
| 8. CMD | # 指定这个容器启动时运行的命令，只有最后一个会生效，可被替代 |
| 9. ENTRYPOINT | # 指定这个容器启动时运行的命令，可以追加命令 |
| 10. ONBUILD | # 当构建一个被继承的dockerfile时，会运行ONBUILD指令，触发指令 |
| 11. COPY | # 类似ADD，将文件拷贝到镜像中 |
| 12. ENV | # 构建的时候设置环境变量 |



FROM	• 它的妈妈是谁（基础镜像）
MAINTAINER	• 告诉别人，你创造了它（维护者信息）
RUN	• 你想让它干啥（把命令前面加上RUN）
ADD	• 往它肚子里放点文件（COPY文件，会自动解压）
WORKDIR	• 我是cd,今天刚化了妆（当前工作目录）
VOLUME	• 给我一个存放行李的地方（目录挂载）
EXPOSE	• 我要打开的门是啥（端口）
RUN	• 奔跑吧，兄弟！（进程要一直运行下去）

🏆 创建一个centos

```
1. [root@localhost ~]# cat mydockerfile
2. FROM centos
3. MAINTAINER liuningle<liuningle@aisino.com>
4. ENV MYPATH /usr/local
5. WORKDIR $MYPATH
6.
7. RUN yum install -y vim
8. RUN yum install -y net-tools
9. EXPOSE 80
10.
11. CMD echo $MYPATH
12. CMD echo "---end---"
13. CMD /bin/bash
14. [root@localhost ~]# docker build -f mydockerfile -t mycentos:1.0 .
15. [root@localhost ~]# docker images
16. REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
17. mycentos             1.0         372e423d8e02     About a minute ago  291MB
18. #Successfully built 372e423d8e02
19. #Successfully tagged mycentos:1.0
20.
21. [root@localhost ~]# docker run -it mycentos:1.0
22. [root@d3d911aea1d2 local]# pwd
23. /usr/local                                     --我们设置的PTAH目录
24. [root@d3d911aea1d2 local]# ifconfig            --net-tools也安装了
25. eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
26.         inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
27.         ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
28.         RX packets 8  bytes 656 (656.0 B)
29.         RX errors 0  dropped 0  overruns 0  frame 0
30.         TX packets 0  bytes 0 (0.0 B)
31.         TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
32.
33. lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
34.         inet 127.0.0.1  netmask 255.0.0.0
35.         loop txqueuelen 1000  (Local Loopback)
36.         RX packets 0  bytes 0 (0.0 B)
37.         RX errors 0  dropped 0  overruns 0  frame 0
38.         TX packets 0  bytes 0 (0.0 B)
39.         TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
40. #我们可通过docker history查看步骤
41. [root@localhost ~]# docker history 372e423d8e02
42. IMAGE          CREATED          CREATED BY          SIZE          COMMENT
43. 372e423d8e02   5 minutes ago   /bin/sh -c #(nop)  CMD ["/bin/sh" "-c" "/bin...  0B
44. d22a7ebee531   5 minutes ago   /bin/sh -c #(nop)  CMD ["/bin/sh" "-c" "echo...  0B
45. 191155b3de4b   5 minutes ago   /bin/sh -c #(nop)  CMD ["/bin/sh" "-c" "echo...  0B
46. 24d9bcc1c30a   5 minutes ago   /bin/sh -c #(nop)  EXPOSE 80          0B
47. 8d10cb5be105   5 minutes ago   /bin/sh -c yum install -y net-tools  23.4MB
48. 4a4e684522d4   5 minutes ago   /bin/sh -c yum install -y vim        58.1MB
49. d256655c9759   6 minutes ago   /bin/sh -c #(nop)  WORKDIR /usr/local  0B
50. 69e98f2ba193   6 minutes ago   /bin/sh -c #(nop)  ENV MYPATH=/usr/local  0B
51. 2417b25aa69e   6 minutes ago   /bin/sh -c #(nop)  MAINTAINER liuningle<liun...  0B
52. 300e315adb2f   2 months ago    /bin/sh -c #(nop)  CMD ["/bin/bash"]    0B
53. <missing>       2 months ago    /bin/sh -c #(nop)  LABEL org.label-schema.sc...  0B
54. <missing>       2 months ago    /bin/sh -c #(nop)  ADD file:bd7a2aed6ede423b7...  209MB
```

通过docker history我们可以了解一个镜像是如何制作的

🏆 比较CMD和ENTRYPOINT

```
1. [root@localhost ~]# vi dockerfile-cmd
2. FROM centos
3. CMD ["ls","-a"]
4. [root@localhost ~]# docker build -f dockerfile-cmd -t centos:2.0 .
5. Sending build context to Docker daemon 190kB
6. Step 1/2 : FROM centos
7. ---> 300e315adb2f
8. Step 2/2 : CMD ["ls","-a"]
9. ---> Running in a378eea0d56e
10. Removing intermediate container a378eea0d56e
11. ---> b1795981cf04
12. Successfully built b1795981cf04
13. Successfully tagged centos:2.0
14. [root@localhost ~]# docker run -it centos:2.0
15. . .dockerenv dev home lib64 media opt root sbin sys usr
16. .. bin etc lib lost+found mnt proc run srv tmp var
17. #此处显示了ls -a命令
18. [root@localhost ~]# docker run -it centos:2.0 -l
19. docker: Error response from daemon: OCI runtime create failed: container_linux.go:370: starting container process
    caused: exec: "-l": executable file not found in $PATH: unknown.
20. ERRO[0001] error waiting for container: context canceled
21. #我们在追加了参数-L的时候发现报错了，因为CMD只会执行最后一个命令，他把-L当成了一个命令
22. [root@localhost ~]# docker run -it centos:2.0 df
23. Filesystem 1K-blocks Used Available Use% Mounted on
24. overlay 17811456 4622068 13189388 26% /
25. tmpfs 65536 0 65536 0% /dev
```

```
1. [root@localhost ~]# vi dockerfile-entrypoint
2. FROM centos
3. ENTRYPOINT ["ls","-a"]
4. [root@localhost ~]# docker build -f dockerfile-entrypoint -t centos:3.0 .
5. Sending build context to Docker daemon 191.5kB
6. Step 1/2 : FROM centos
7. ---> 300e315adb2f
8. Step 2/2 : ENTRYPOINT ["ls","-a"]
9. ---> Running in d80ec7aedd11
10. Removing intermediate container d80ec7aedd11
11. ---> bf83544aa006
12. Successfully built bf83544aa006
13. Successfully tagged centos:3.0
14. [root@localhost ~]# docker run -it centos:3.0 -l
15. total 0
16. drwxr-xr-x. 1 root root 6 Feb 20 16:49 .
17. drwxr-xr-x. 1 root root 6 Feb 20 16:49 ..
18. -rwxr-xr-x. 1 root root 0 Feb 20 16:49 .dockerenv
19. lrwxrwxrwx. 1 root root 7 Nov 3 15:22 bin -> usr/bin
20. drwxr-xr-x. 5 root root 360 Feb 20 16:49 dev
21. drwxr-xr-x. 1 root root 66 Feb 20 16:49 etc
22. #此时发现能用-L了，因为entrypoint是追加了参数-L
23. [root@localhost ~]# docker run -it centos:3.0 df
24. ls: cannot access 'df': No such file or directory
```

🏆 实战：Tomcat镜像

- 1.准备镜像文件tomcat压缩包，jdk压缩包
- 2.编写dockerfile文件，官方命名Dockerfile，==build时会自动寻找这个文件，就不需要-f指定了==



```
1. FROM centos
2. MAINTAINER liuningle<liuningle@aisino.com>
3.
4. COPY readme.txt /usr/local/readme.txt
5.
6. ADD jdk-15.0.2_linux-x64_bin.tar.gz /usr/local
7. ADD apache-tomcat-9.0.43.tar.gz /usr/local
8.
9. RUN yum install -y vim
10.
11. ENV MYPATH /usr/local
12. WORKDIR $MYPATH
13.
14. ENV JAVA_HOME /usr/local/jdk-15.0.2
15. ENV CLASSPATH $JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib/tools.jar
16. ENV CATALINA_HOME /usr/local/apache-tomcat-9.0.43
17. ENV CATALINA_BASH /usr/local/apache-tomcat-9.0.43
18. ENV PATH $PATH:$JAVA_HOME/bin:$CATALINA_HOME/lib:$CATALINA_HOME/bin
19.
20. EXPOSE 8080
21.
22. CMD /usr/local/apache-tomcat-9.0.43/bin/startup.sh && tail -F /usr/local/apache-tomcat-9.0.43/logs/catalina.out
```

3.构建镜像



```
1. [root@localhost tomcat]# docker build -f 1 -t lnltomcat:02 .
2. Sending build context to Docker daemon 353.4MB
3. Step 1/15 : FROM centos
4. ----> 300e315adb2f
5. Step 2/15 : MAINTAINER liuningle<liuningle@aisino.com>
6. ----> Using cache
7. ----> 2417b25aa69e
8. Step 3/15 : COPY readme.txt /usr/local/readme.txt
9. ----> Using cache
10. ----> 5a3b06bdbe2d
11. Step 4/15 : ADD jdk-15.0.2_linux-x64_bin.tar.gz /usr/local
12. ----> 323152020842
13. Step 5/15 : ADD apache-tomcat-9.0.43.tar.gz /usr/local
14. ----> 90ab12eb9304
15. Step 6/15 : RUN yum install -y vim
16. ----> Running in d7de62a6ea46
17. CentOS Linux 8 - AppStream          549 kB/s | 6.3 MB    00:11
18. CentOS Linux 8 - BaseOS             610 kB/s | 2.3 MB    00:03
19. CentOS Linux 8 - Extras             4.9 kB/s | 8.6 kB    00:01
20. Last metadata expiration check: 0:00:01 ago on Sat Feb 20 20:05:27 2021.
21. Dependencies resolved.
22. =====
23. Package                Arch          Version           Repository        Size
24. =====
25. Installing:
26. vim-enhanced            x86_64        2:8.0.1763-15.el8 appstream         1.4 M
27. Installing dependencies:
28. gpm-libs                x86_64        1.20.7-15.el8     appstream         39 k
29. vim-common              x86_64        2:8.0.1763-15.el8 appstream         6.3 M
30. vim-filesystem          noarch        2:8.0.1763-15.el8 appstream         48 k
31. which                   x86_64        2.21-12.el8       baseos            49 k
32.
33. Transaction Summary
34. =====
35. Install 5 Packages
36.
37. Total download size: 7.8 M
38. Installed size: 30 M
39. Downloading Packages:
40. (1/5): gpm-libs-1.20.7-15.el8.x86_64.rpm    121 kB/s | 39 kB    00:00
41. (2/5): vim-filesystem-8.0.1763-15.el8.noarch.rp 116 kB/s | 48 kB    00:00
42. (3/5): which-2.21-12.el8.x86_64.rpm         82 kB/s | 49 kB    00:00
43. (4/5): vim-enhanced-8.0.1763-15.el8.x86_64.rpm 345 kB/s | 1.4 MB   00:04
44. (5/5): vim-common-8.0.1763-15.el8.x86_64.rpm 465 kB/s | 6.3 MB   00:13
45. -----
46. Total                                531 kB/s | 7.8 MB   00:15
47. warning: /var/cache/dnf/appstream-02e86d1c976ab532/packages/gpm-libs-1.20.7-15.el8.x86_64.rpm: Header V3
    RSA/SHA256 Signature, key ID 8483c65d: NOKEY
48. CentOS Linux 8 - AppStream          1.6 MB/s | 1.6 kB    00:00
49. Importing GPG key 0x8483C65D:
50. Userid      : "CentOS (CentOS Official Signing Key) <security@centos.org>"
51. Fingerprint: 99DB 70FA E1D7 CE22 7FB6 4882 05B5 55B3 8483 C65D
52. From        : /etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
53. Key imported successfully
54. Running transaction check
55. Transaction check succeeded.
56. Running transaction test
57. Transaction test succeeded.
58. Running transaction
59. Preparing      :                                1/1
60. Installing    : which-2.21-12.el8.x86_64        1/5
61. Installing    : vim-filesystem-2:8.0.1763-15.el8.noarch 2/5
62. Installing    : vim-common-2:8.0.1763-15.el8.x86_64    3/5
63. Installing    : gpm-libs-1.20.7-15.el8.x86_64        4/5
64. Running scriptlet: gpm-libs-1.20.7-15.el8.x86_64    4/5
65. Installing    : vim-enhanced-2:8.0.1763-15.el8.x86_64 5/5
66. Running scriptlet: vim-enhanced-2:8.0.1763-15.el8.x86_64 5/5
```

```
67.   Running scriptlet: vim-common-2:8.0.1763-15.el8.x86_64          5/5
68.   Verifying       : gpm-libs-1.20.7-15.el8.x86_64                1/5
69.   Verifying       : vim-common-2:8.0.1763-15.el8.x86_64          2/5
70.   Verifying       : vim-enhanced-2:8.0.1763-15.el8.x86_64        3/5
71.   Verifying       : vim-filesystem-2:8.0.1763-15.el8.noarch       4/5
72.   Verifying       : which-2.21-12.el8.x86_64                     5/5
73.
74. Installed:
75.   gpm-libs-1.20.7-15.el8.x86_64      vim-common-2:8.0.1763-15.el8.x86_64
76.   vim-enhanced-2:8.0.1763-15.el8.x86_64 vim-filesystem-2:8.0.1763-15.el8.noarch
77.   which-2.21-12.el8.x86_64
78.
79. Complete!
80. Removing intermediate container d7de62a6ea46
81. ---> 1588f8d7540c
82. Step 7/15 : ENV MYPATH /usr/local
83. ---> Running in f675d51eb669
84. Removing intermediate container f675d51eb669
85. ---> ea3e1cf22146
86. Step 8/15 : WORKDIR $MYPATH
87. ---> Running in ffd52af5223b
88. Removing intermediate container ffd52af5223b
89. ---> dd7eb7695d2d
90. Step 9/15 : ENV JAVA_HOME /usr/local/jdk-15.0.2
91. ---> Running in 083ea6aa93dc
92. Removing intermediate container 083ea6aa93dc
93. ---> 82dd37ab242c
94. Step 10/15 : ENV CLASSPATH $JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib/tools.jar
95. ---> Running in c7edfae89295
96. Removing intermediate container c7edfae89295
97. ---> cff8f1250658
98. Step 11/15 : ENV CATALINA_HOME /usr/local/apache-tomcat-9.0.43
99. ---> Running in 52643827841a
100. Removing intermediate container 52643827841a
101. ---> 8317bc605ce0
102. Step 12/15 : ENV CATALINA_BASH /usr/local/apache-tomcat-9.0.43
103. ---> Running in 72d596338a7b
104. Removing intermediate container 72d596338a7b
105. ---> d1d373868c1e
106. Step 13/15 : ENV PATH $PATH:$JAVA_HOME/bin:$CATALINA_HOME/lib:$CATALINA_HOME/bin
107. ---> Running in 973c6f7960ee
108. Removing intermediate container 973c6f7960ee
109. ---> 517aeee5614c
110. Step 14/15 : EXPOSE 8080
111. ---> Running in e48634333c5d
112. Removing intermediate container e48634333c5d
113. ---> f01236ad4f1f
114. Step 15/15 : CMD /usr/local/apache-tomcat-9.0.43/bin/startup.sh && tail -F /usr/local/apache-tomcat-
    9.0.43/logs/catalina.out
115. ---> Running in df10a46ade44
116. Removing intermediate container df10a46ade44
117. ---> 3bfe8e4b7690
118. Successfully built 3bfe8e4b7690
119. Successfully tagged lnltomcat:02
```

4.启动镜像

1. # `docker run -d -p 8080:8080 --name lnltomcat -v /root/tomcat/test:/usr/Local/apache-tomcat-9.0.43/webapps/test`
`v /root/tomcat/tomcatLogs/:/usr/Loacl/apache-tomcat-9.0.43/Logs lnltomcat:02`
2. #为防止踩坑，请务必使用docker logs [容器ID] 来检查错误！
3. #踩坑1：脚本编写错误 踩坑2：jdk版本太低

5.访问测试

```
1. [root@localhost tomcat]# docker ps
2. CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
   NAMES
3. 5d856a89c29f   lnltomcat:02        "/bin/sh -c '/usr/lo..." 4 minutes ago   Up 4 minutes   0.0.0.0:8080->8080/tcp
   lnltomcat02
4. [root@localhost tomcat]# docker exec -it 5d856a89c29f /bin/bash
5. [root@5d856a89c29f local]# ls
6. apache-tomcat-9.0.43  bin  etc  games  include  jdk-15.0.2  lib  lib64  libexec  readme.txt  sbin  share  src
```



6.发布项目

```
1. [root@bogon WEB-INF]# cat /root/tomcat/test/WEB-INF/web.xml
2. <?xml version="1.0" encoding="UTF-8"?>
3. <web-app xmlns="http://java.sun.com/xml/ns/javaee"
4.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.         xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
6.                             http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
7.         version="2.5">
8.
9. </web-app>
```



```
1. [root@bogon test]# cat /root/tomcat/test/index.jsp
2. <%@ page language="java" contentType="text/html; charset=UTF-8"
3.     pageEncoding="UTF-8"%>
4. <!DOCTYPE html>
5. <html>
6. <head>
7. <meta charset="utf-8">
8. <title>hello,lele</title>
9. </head>
10. <body>
11. Hello World!<br/>
12. <%
13. System.out.println("---my test---");
14. %>
15. </body>
16. </html>
```



web访问: <http://172.31.202.45:8080/test/>

```
1. [root@bogon tomcatlogs]# docker logs lnltomcat02
2. ---my test---
3. ---my test---
4. ---my test---
```



发布自己的镜像

- 1.登录至dockerhub, 注册自己的账号, 我的账号是liuningle
- 2.确定账号可以登录
- 3.在我们的服务器上提交自己的镜像



```
1. [root@bogon WEB-INF]# docker login --help
2.
3.
4. Usage:  docker login [OPTIONS] [SERVER]
5.
6. Log in to a Docker registry.
7. If no server is specified, the default is defined by the daemon.
8.
9. Options:
10.  -p, --password string    Password
11.      --password-stdin      Take the password from stdin
12.  -u, --username string     Username
13. [root@bogon WEB-INF]# docker login -u liuningle
14. Password:
15. WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
16. Configure a credential helper to remove this warning. See
17. https://docs.docker.com/engine/reference/commandline/login/#credentials-store
18.
19. Login Succeeded
```

4.登录成功可以提交镜像了。docker push



```
1. #push自己的镜像到服务器上
2. [root@bogon WEB-INF]# docker push lnltomcat:02
3. The push refers to repository [docker.io/library/lnltomcat]
4. 58aa9ef42505: Preparing
5. 89a108e5d111: Preparing
6. a83eb93886f0: Preparing
7. 825b764d678c: Preparing
8. 2653d992f4ef: Preparing
9. denied: requested access to the resource is denied#
10.
11. #push镜像的问题？
12. [root@bogon WEB-INF]# docker images
13. REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
14. ln1/lnltomcat        1.0          3bfe8e4b7690     3 days ago       612MB
15. [root@bogon WEB-INF]# docker push ln1/lnltomcat:1.0
16. The push refers to repository [docker.io/ln1/lnltomcat]
17. 58aa9ef42505: Preparing
18. 89a108e5d111: Preparing
19. a83eb93886f0: Preparing
20. 825b764d678c: Preparing
21. 2653d992f4ef: Preparing
22. denied: requested access to the resource is denied
23. #也失败了，这是因为上传镜像，需要将自己docker user添加进去
24. [root@bogon WEB-INF]# docker tag --help
25.
26. Usage:  docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
27.
28. Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
29. [root@bogon WEB-INF]# docker tag lnltomcat:02 liuningle/lnltomcat:02
30. [root@bogon WEB-INF]# docker push liuningle/lnltomcat:02
31. The push refers to repository [docker.io/liuningle/lnltomcat]
32. 58aa9ef42505: Pushing [=====>] 33.63MB/58.08MB
33. 89a108e5d111: Pushed
34. a83eb93886f0: Pushing [===>] 22.77MB/329MB
35. 825b764d678c: Pushed
36. 2653d992f4ef: Pushing [=====>] 42.09MB/209.3MB
37. ^C
38. #我们用docker tag命令将镜像改为 user+名字格式，即可正常上传。而且我们发现，docker镜像上传也是一层一层的上传
```

🏆 理解docker0



```
1. #先将所有镜像删除
2. [root@bogon WEB-INF]# docker rmi -f $(docker images -a -q)
3. [root@bogon WEB-INF]# ip addr
4. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
5.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
6.     inet 127.0.0.1/8 scope host lo
7.         valid_lft forever preferred_lft forever
8.     inet6 ::1/128 scope host
9.         valid_lft forever preferred_lft forever
10. 2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
11.     link/ether 00:0c:29:fc:57:e2 brd ff:ff:ff:ff:ff:ff
12.     inet 172.31.202.45/24 brd 172.31.202.255 scope global noprefixroute ens33
13.         valid_lft forever preferred_lft forever
14.     inet6 fe80::20c:29ff:fe57:e2/64 scope link
15.         valid_lft forever preferred_lft forever
16. 3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
17.     link/ether 02:42:98:04:68:a9 brd ff:ff:ff:ff:ff:ff
18.     inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
19.         valid_lft forever preferred_lft forever
20.     inet6 fe80::42:98ff:fe04:68a9/64 scope link
21.         valid_lft forever preferred_lft forever
22. #docker0是docker默认的网卡
```

🏆 原理



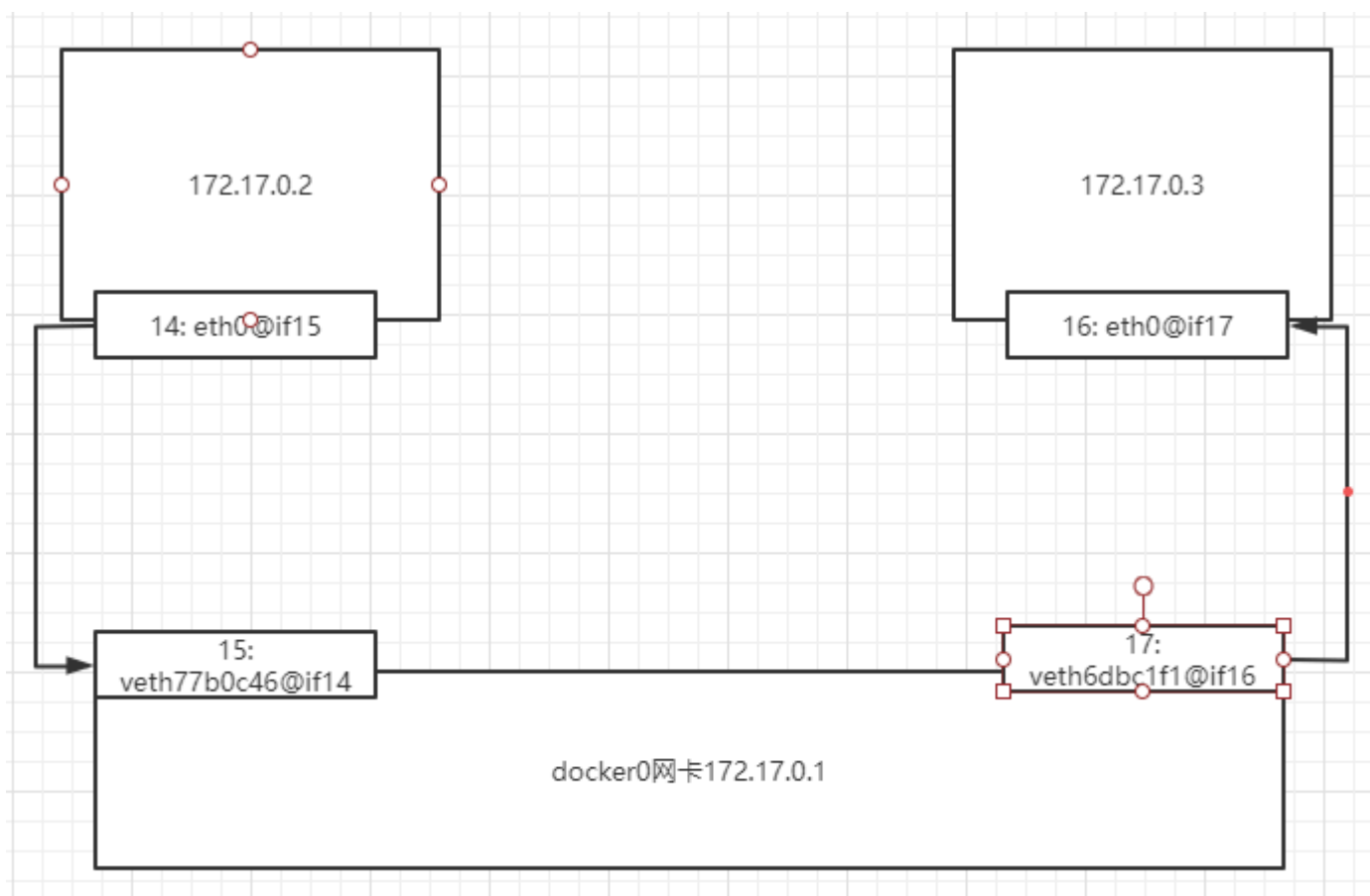
```
1. [root@bogon /]# docker run -d -P --name tomcat01 tomcat
2. 83b4703873b5213054e7e506e721a5d68f1ca40521ac68b363616a7f333fa981
3. #查看容器内ip
4. [root@bogon /]# docker exec -it tomcat01 ip addr
5. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
6.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
7.     inet 127.0.0.1/8 scope host lo
8.         valid_lft forever preferred_lft forever
9. 14: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
10.     link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
11.     inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
12.         valid_lft forever preferred_lft forever
13. [root@bogon /]# docker run -d -P --name tomcat02 tomcat
14. 56bb3fef856fa86b39bd862c5227893fad81b2cc64daaabd67f54680e6708782
15. [root@bogon /]# docker exec -it tomcat02 ip addr
16. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
17.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
18.     inet 127.0.0.1/8 scope host lo
19.         valid_lft forever preferred_lft forever
20. 16: eth0@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
21.     link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
22.     inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
23.         valid_lft forever preferred_lft forever
24. #发现Linux能ping通容器内部
25. [root@bogon /]# ping 172.17.0.3
26. PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
27. 64 bytes from 172.17.0.3: icmp_seq=1 ttl=64 time=0.207 ms
28. 64 bytes from 172.17.0.3: icmp_seq=2 ttl=64 time=0.065 ms
```

1.每启动一个docker容器，docker就会给docker容器分配一个ip，只要安装了docker，就会分配一个docker0网卡

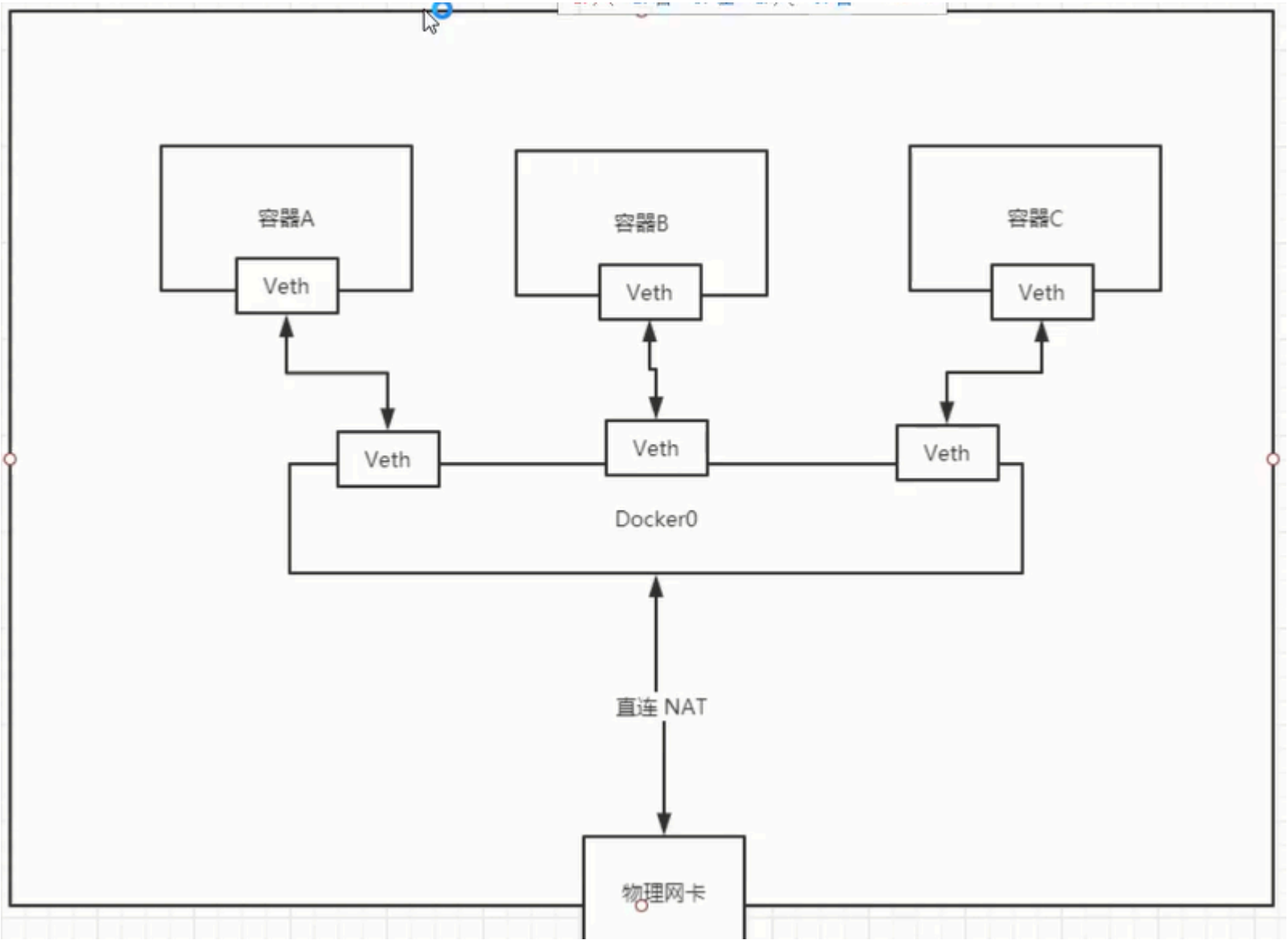
2.当启动一个容器测试，发现又多一对网卡



```
1. [root@bogon /]# ip addr
2. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
3.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4.     inet 127.0.0.1/8 scope host lo
5.         valid_lft forever preferred_lft forever
6.     inet6 ::1/128 scope host
7.         valid_lft forever preferred_lft forever
8. 2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
9.     link/ether 00:0c:29:fc:57:e2 brd ff:ff:ff:ff:ff:ff
10.    inet 172.31.202.45/24 brd 172.31.202.255 scope global noprefixroute ens33
11.        valid_lft forever preferred_lft forever
12.    inet6 fe80::20c:29ff:fe57:e2/64 scope link
13.        valid_lft forever preferred_lft forever
14. 3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
15.     link/ether 02:42:98:04:68:a9 brd ff:ff:ff:ff:ff:ff
16.     inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
17.         valid_lft forever preferred_lft forever
18.     inet6 fe80::42:98ff:fe04:68a9/64 scope link
19.         valid_lft forever preferred_lft forever
20. 15: veth77b0c46@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group
    default
21.     link/ether 22:b3:44:bf:da:48 brd ff:ff:ff:ff:ff:ff link-netnsid 0
22.     inet6 fe80::20b3:44ff:feb3:da48/64 scope link
23.         valid_lft forever preferred_lft forever
24. 17: veth6dbc1f1@if16: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group
    default
25.     link/ether be:44:4f:0f:dd:ea brd ff:ff:ff:ff:ff:ff link-netnsid 1
26.     inet6 fe80::bc44:4fff:fe0f:ddea/64 scope link
27.         valid_lft forever preferred_lft forever
28. #我们发现这个容器带来网卡都是一对一对的
29.
30. #veth pair就是一对的虚拟设备接口，都是成对出现的，一段连着协议，一段连着彼此
31. #测试后容器1、2都互通，因为都是通过172.17.0.1桥接
```



docke使用的linux桥接，如下



只要容器一删除，对应的一对网桥就没有了

🏆 —link

```
1. #有些时候，我们面临服务ip更换情况，该如何通过服务名字访问容器？
2. [root@bogon /]# docker exec -it tomcat02 ping tomcat01
3. ping: tomcat01: Name or service not known
4. #发现名字不通，通过--Link可以解决
5. [root@bogon /]# docker run -d -P --name tomcat03 --link tomcat02 tomcat
6. a88d674661576633a7a4431e8adbe78fc8352a7f1802b1681b950b6b01a98140
7. [root@bogon /]# docker exec -it tomcat03 ping tomcat02
8. PING tomcat02 (172.17.0.3) 56(84) bytes of data.
9. 64 bytes from tomcat02 (172.17.0.3): icmp_seq=1 ttl=64 time=0.124 ms
10. 64 bytes from tomcat02 (172.17.0.3): icmp_seq=2 ttl=64 time=0.075 ms
11. #我们发现可以ping通，那么反向可以ping通吗？
12. [root@bogon /]# docker exec -it tomcat02 ping tomcat03
13. ping: tomcat03: Name or service not known
14. #ping不通
15. #我们通过docker inspect查看容器网卡内容
16. #其实是tomcat03本地配置了一个tomcat02的配置，而tomcat02并没有配置
17. [root@bogon /]# docker exec -it tomcat03 cat /etc/hosts
18. 127.0.0.1      localhost
19. ::1           localhost ip6-localhost ip6-loopback
20. fe00::0 ip6-localnet
21. ff00::0 ip6-mcastprefix
22. ff02::1 ip6-allnodes
23. ff02::2 ip6-allrouters
24. 172.17.0.3     tomcat02 56bb3fef856f #此处为配置的容器2的ip
25. 172.17.0.4     a88d67466157
```

—link其实是在hosts中配置了一个172.17.0.3的名字映射，docker0并不支持容器名访问

🏆 自定义网络

查看所有网络



```
1. [root@bogon /]# docker network --help
2.
3. Usage:  docker network COMMAND
4.
5. Manage networks
6.
7. Commands:
8.   connect    Connect a container to a network
9.   create     Create a network
10.  disconnect  Disconnect a container from a network
11.  inspect     Display detailed information on one or more networks
12.  ls         List networks
13.  prune      Remove all unused networks
14.  rm         Remove one or more networks
15.
16. Run 'docker network COMMAND --help' for more information on a command.
17. [root@bogon /]# docker network ls
18. NETWORK ID          NAME           DRIVER          SCOPE
19. 22afa40fd7c0        bridge        bridge          local
20. 459c2f2fb892        host          host            local
21. a2db766f6b7d        none         null            local
```

网络模式

bridge: 桥接 (默认) none: 不配置网络 host: 和宿主机共享网络 container: 容器网络连通 (少用)



```
1. #我们直接启动命令 --netwok bridge
2. docker run -d -P --name tomcat01 tomcat
3. docker run -d -P --name tomcat01 --net bridge tomcat
4. #docker0特点:默认,但是域名不能访问,--Link可以打通连接
5. #我们可以定义一个网络
6. [root@bogon /]# docker network create --driver bridge --subnet 192.168.0.0/16 --gateway 192.168.0.1 mynet
7. 0f87b54b4ddb7fb5176cf6374eba394137bc779e371cbcf1ff48725cf63b7e3c
8. [root@bogon /]# docker network ls
9. NETWORK ID          NAME           DRIVER         SCOPE
10. 22afa40fd7c0         bridge        bridge         local
11. 459c2f2fb892         host          host           local
12. 0f87b54b4ddb         mynet         bridge         local
13. a2db766f6b7d         none          null           local
14. #发现我们增加的一张网卡mynet有了
15. #查看我们自己的网卡
16. [root@bogon /]# docker network inspect mynet
17. [
18.     {
19.         "Name": "mynet",
20.         "Id": "0f87b54b4ddb7fb5176cf6374eba394137bc779e371cbcf1ff48725cf63b7e3c",
21.         "Created": "2021-02-23T21:02:43.683566918Z",
22.         "Scope": "local",
23.         "Driver": "bridge",
24.         "EnableIPv6": false,
25.         "IPAM": {
26.             "Driver": "default",
27.             "Options": {},
28.             "Config": [
29.                 {
30.                     "Subnet": "192.168.0.0/16",
31.                     "Gateway": "192.168.0.1"
32.                 }
33.             ]
34.         },
35.         "Internal": false,
36.         "Attachable": false,
37.         "Ingress": false,
38.         "ConfigFrom": {
39.             "Network": ""
40.         },
41.         "ConfigOnly": false,
42.         "Containers": {},
43.         "Options": {},
44.         "Labels": {}
45.     }
46. ]
47. #指定子网创建容器
48. [root@bogon /]# docker run -d -P --name tomcat-net-01 --net mynet tomcat
49. fec147df5306fefca7351a4dbc468f57fe2b6bbf0c42ae149faea542ba6f7c7f
50. [root@bogon /]# docker run -d -P --name tomcat-net-02 --net mynet tomcat
51. f71e3b62d14cc55d227741f245408ea2f4e3c08bd189daa340a2dbc1dc5b5dfe
52. [root@bogon /]# docker network inspect mynet
53.     "Containers": {
54.         "f71e3b62d14cc55d227741f245408ea2f4e3c08bd189daa340a2dbc1dc5b5dfe": {
55.             "Name": "tomcat-net-02",
56.             "EndpointID": "8d4405c2b71e935eae8f9d7036654d53c46907660a92bcc73676ce6f1ff47796",
57.             "MacAddress": "02:42:c0:a8:00:03",
58.             "IPv4Address": "192.168.0.3/16",
59.             "IPv6Address": ""
60.         },
61.         "fec147df5306fefca7351a4dbc468f57fe2b6bbf0c42ae149faea542ba6f7c7f": {
62.             "Name": "tomcat-net-01",
63.             "EndpointID": "fdd6d46a987477ef5abb882c6ec19831c6dc4c6830909448537901056e6d5ce4",
64.             "MacAddress": "02:42:c0:a8:00:02",
65.             "IPv4Address": "192.168.0.2/16",
66.             "IPv6Address": ""
67.         }
68.     }
```



```
68.     },
69. #我们发现了自己创建的容器的ip , 开始测试ping
70. [root@bogon /]# docker exec -it tomcat-net-01 ping 192.168.0.3
71. PING 192.168.0.3 (192.168.0.3) 56(84) bytes of data.
72. 64 bytes from 192.168.0.3: icmp_seq=1 ttl=64 time=0.113 ms
73. 64 bytes from 192.168.0.3: icmp_seq=2 ttl=64 time=0.079 ms
74. [root@bogon /]# docker exec -it tomcat-net-01 ping tomcat-net-02
75. PING tomcat-net-02 (192.168.0.3) 56(84) bytes of data.
76. 64 bytes from tomcat-net-02.mynet (192.168.0.3): icmp_seq=1 ttl=64 time=0.065 ms
77. 64 bytes from tomcat-net-02.mynet (192.168.0.3): icmp_seq=2 ttl=64 time=0.081 ms
78. #我们发现互相连通 , 并可以访问服务
```

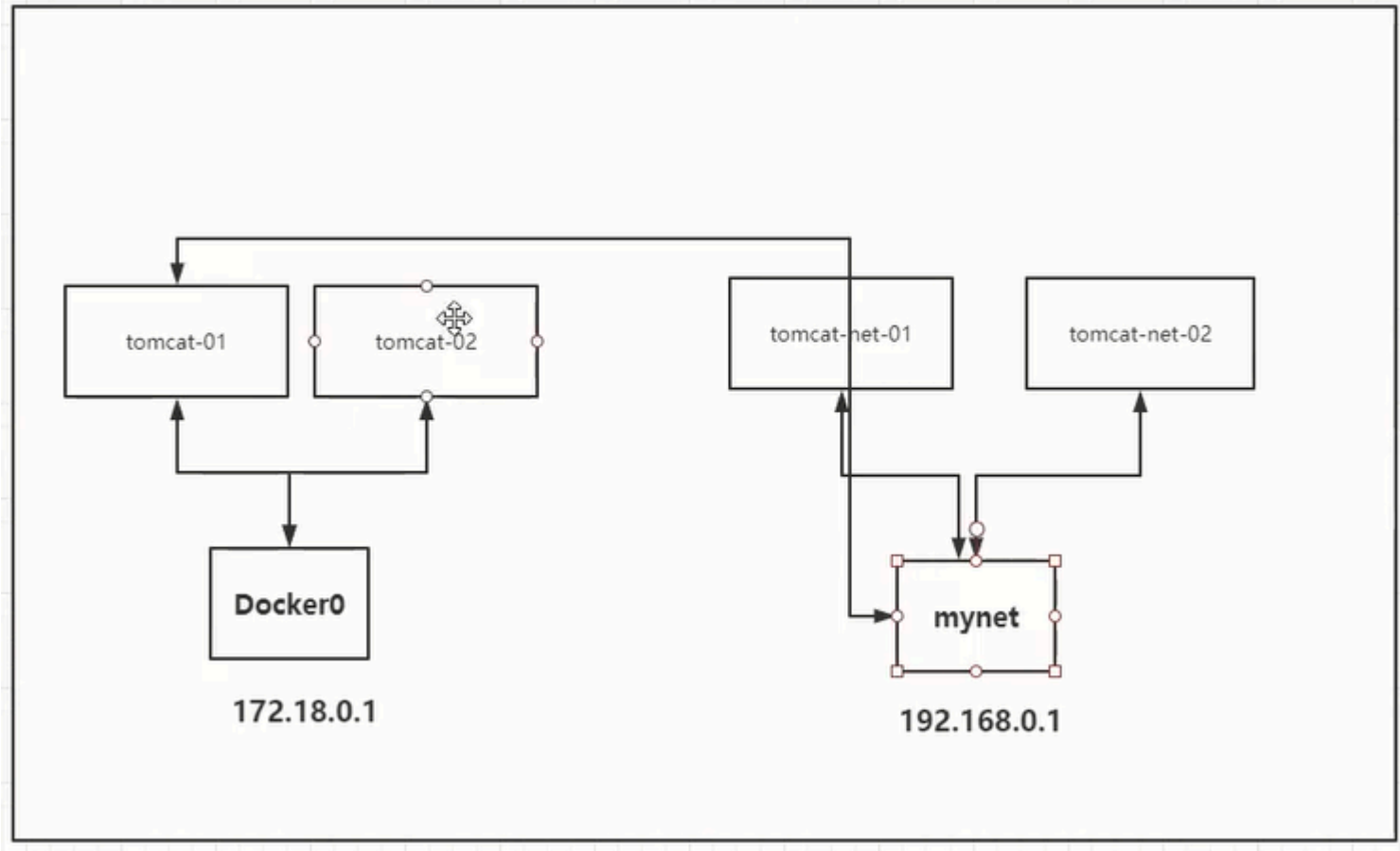
自建的网络docker已经维护好了对应关系, 推荐平时这么连

🏆 网络连接

```
1. #使用docker network connect使容器桥接到另一张网卡 , 因为此时tomcat01其实是桥接在了docker0上的
2. [root@bogon /]# docker network connect mynet tomcat01
3. [root@bogon /]# docker exec -it tomcat02 ping 192.168.0.1
4. PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
5. 64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=0.121 ms
6. 64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=0.059 ms
7. 64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=0.080 ms
8. 64 bytes from 192.168.0.1: icmp_seq=4 ttl=64 time=0.106 ms
9. ^C
10. --- 192.168.0.1 ping statistics ---
11. 4 packets transmitted, 4 received, 0% packet loss, time 3ms
12. rtt min/avg/max/mdev = 0.059/0.091/0.121/0.025 ms
13. [root@bogon /]# docker exec -it tomcat02 ip addr
14. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
15.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
16.     inet 127.0.0.1/8 scope host lo
17.         valid_lft forever preferred_lft forever
18. 16: eth0@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
19.     link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
20.     inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
21.         valid_lft forever preferred_lft forever
22. #其实此时就相当于tomcat01有两个ip , 一个公网一个私网
23. [root@bogon /]# docker exec -it tomcat01 ip addr
24. 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
25.     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
26.     inet 127.0.0.1/8 scope host lo
27.         valid_lft forever preferred_lft forever
28. 14: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
29.     link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
30.     inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
31.         valid_lft forever preferred_lft forever
32. 25: eth1@if26: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
33.     link/ether 02:42:c0:a8:00:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
34.     inet 192.168.0.4/16 brd 192.168.255.255 scope global eth1
35.         valid_lft forever preferred_lft foreve
```

```
"Containers": {
  "83b4703873b5213054e7e506e721a5d68f1ca40521ac68b363616a7f333fa981": {
    "Name": "tomcat01",
    "EndpointID": "bcaaf2f715d34b9ff0f9c3b8bfbd54d768e3ba73a2d68d1559b44cc2d4d5a92",
    "MacAddress": "02:42:c0:a8:00:04",
    "IPv4Address": "192.168.0.4/16",
    "IPv6Address": ""
  },
  "f71e3b62d14cc55d227741f245408ea2f4e3c08bd189daa340a2dbc1dc5b5dfe": {
    "Name": "tomcat-net-02",
    "EndpointID": "8d4405c2b71e935eae8f9d7036654d53c46907660a92bcc73676ce6f1ff47796",
    "MacAddress": "02:42:c0:a8:00:03",
    "IPv4Address": "192.168.0.3/16",
    "IPv6Address": ""
  },
  "fec147df5306fefca7351a4dbc468f57fe2b6bbf0c42ae149faea542ba6f7c7f": {
    "Name": "tomcat-net-01",
    "EndpointID": "fdd6d46a987477ef5abb882c6ec19831c6dc4c6830909448537901056e6d5ce4",
    "MacAddress": "02:42:c0:a8:00:02",
    "IPv4Address": "192.168.0.2/16",
    "IPv6Address": ""
  }
}
```

1. [root@bogon /]# docker exec -it tomcat01 ping tomcat-net-01
2. PING tomcat-net-01 (192.168.0.2) 56(84) bytes of data.
3. 64 bytes from tomcat-net-01.mynet (192.168.0.2): icmp_seq=1 ttl=64 time=0.136 ms
4. 64 bytes from tomcat-net-01.mynet (192.168.0.2): icmp_seq=2 ttl=64 time=0.070 ms
5. ^C
6. --- tomcat-net-01 ping statistics ---
7. 2 packets transmitted, 2 received, 0% packet loss, time 1ms
8. rtt min/avg/max/mdev = 0.070/0.103/0.136/0.033 ms
9. [root@bogon /]# docker exec -it tomcat02 ping tomcat-net-01
10. ping: tomcat-net-01: Name or service not known
11. #因为02没有打通到mynet网卡 所以不通！



结论：假设要跨网络操作，需要使用docker network connect连通



Docker Compose

版权声明：本文为博主原创文章，遵循CC 4.0 BY-SA版权协议, 转载请附上原文出处链接和本声明，KuangStudy, 以学为伴，一生相伴！
本文链接：<https://www.kuangstudy.com/bbs/1366344087514509313>

嘿~ 大神，别默默的看了，快来点评一下吧！

总共已有 3 条评论

✍ 提交评论



千里之行，始于足下

👤 回复 2021-10-11 22:19:09

兄弟很强，总结的很好



村长

👤 回复 2021-03-06 10:33:41

好东西，怎么才能保存下来？



八重樱

👤 回复 2021-03-01 20:07:48

这不愁没钱买狂神的笔记，免费的就来了😁

[没有更多了 \(共 3 条 / 当前 1 页\)](#)



公众号



官方抖音